

MI DISP API

Version 2.26

© 2022 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
2.03	Initial release	04/12/2018
2.04	- 新增 API - MI_DISP_GetLcdParam - MI_DISP_SetLcdParam - MI_DISP_DeviceGetColorTempature - MI_DISP_DeviceSetColorTempature - MI_DISP_DeviceSetGammaParam - MI_DISP_SetVideoLayerRotateMode - 数据结构修改 - MI_DISP_InputPortAttr_t - 增加参数: - MI_U16 u16SrcWidth; - MI_U16 u16SrcHeight;	06/06/2019
2.05	- 新增 API - MI_DISP_DeviceAttach - MI_DISP_DeviceDetach	06/19/2019
2.06	- 修改 MI_DISP_ClearInputPortBuffer 参数 - 新增 API - MI_DISP_InitDev - MI_DISP_DeInitDev	05/13/2020
2.07	新增 Interface Type	06/18/2020
2.08	- 修改芯片名称 - 表 2 -1 删除 MI_DISP_RefreshInputPort - 新增 API: MI_DISP_SetZoomInWindow	08/04/2020
2.09	- 新增接口类型 - E_MI_DISP_INTF_TTL_SPI_IF	09/03/2020
2.10	- 新增接口类型 - E_MI_DISP_INTF_SRGB	10/26/2020
2.11	- 新增数据类型 - MI_DISP_DevAttachAttr_t	12/24/2020
2.12	- 新增 API - MI_DISP_SetWBCSource - MI_DISP_GetWBCSource - MI_DISP_SetWBCAttr - MI_DISP_GetWBCAttr - MI_DISP_EnableWBC - MI_DISP_DisableWBC	01/05/2021
2.13	- 新增接口类型 - E_MI_DISP_INTF_MCU	01/22/2021

Revision No.	Description	Date
	E_MI_DISP_INTF_MCU_NOFLM	
2.14	- 修改 struct MI_DISP_SyncInfo_t - 新增参数 - MI_U16 u16VStart; - MI_U16 u16HStart;	04/16/2021
2.15	- 新增时序类型 - E_MI_DISP_OUTPUT_1920x1080_5994 - E_MI_DISP_OUTPUT_1920x1080_2997 - E_MI_DISP_OUTPUT_1280x720_5994	04/22/2021
2.16	- 新增时序类型 - E_MI_DISP_OUTPUT_1280x720_2997	05/17/2021
2.17	新增 Muffin 相关内容	05/25/2021
2.18	- 新增时序类型 - E_MI_DISP_OUTPUT_2560x1440_50 - E_MI_DISP_OUTPUT_2560x1440_60 - E_MI_DISP_OUTPUT_3840x2160_25	06/10/2021
2.19	- 新增时序类型，修改 Ikayaki 框图 - E_MI_DISP_OUTPUT_3840x2160_2997	08/05/2021
	Added PROCFS introduction	08/25/2021
2.20	- 新增 Mochi 相关内容 - 错误码优化	09/03/2021
2.21	- 新增支援 BT601 输出 - 新增支援 CVBS 接口下的 csc	10/12/2021
2.22	- 新增时序类型 - E_MI_DISP_OUTPUT_720P24 - E_MI_DISP_OUTPUT_720P25 - E_MI_DISP_OUTPUT_720P30 - E_MI_DISP_OUTPUT_1920x1080_2398 - E_MI_DISP_OUTPUT_3840x2160_24 - E_MI_DISP_OUTPUT_848x480_60 - E_MI_DISP_OUTPUT_1280x768_60 - E_MI_DISP_OUTPUT_1280x960_60 - E_MI_DISP_OUTPUT_1360x768_60 - E_MI_DISP_OUTPUT_1400x1050_60 - E_MI_DISP_OUTPUT_1600x900_60 - E_MI_DISP_OUTPUT_1920x1440_60	11/29/2021
2.23	- 新增接口类型 - E_MI_DISP_INTF_BT1120_DDR	12/08/2021
2.24	新增接口	12/20/2021

Revision No.	Description	Date
	• MI_DISP_SetVideoLayerAttrBatchBegin • MI_DISP_SetVideoLayerAttrBatchEnd • 修改以下接口注意事项 • MI_DISP_DisableVideoLayer • MI_DISP_UnBindVideoLayer	
2.25	• 修改 1.2.6 流程框图 • 修改 3.23 注意事项	2/16/2022
2.26	• 新增 Maruko 相关内容	3/7/2022

TABLE OF CONTENTS

REVISION HISTORY	i
TABLE OF CONTENTS	iv
1. 概述	1
1.1. 模块说明	1
1.2. 流程框图	1
1.2.1. Taiyaki/Takoyaki	1
1.2.2. Pretzel/Pudding	2
1.2.3. Tiramisu	3
1.2.4. Ikayaki	4
1.2.5. Muffin	5
1.2.6. Mochi	8
1.2.7. Maruko	9
1.3. 关键字说明	9
2. API 参考	11
2.1. MI_DISP_Enable	13
2.2. MI_DISP_Disable	14
2.3. MI_DISP_SetPubAttr	15
2.4. MI_DISP_GetPubAttr	16
2.5. MI_DISP_DeviceAttach	16
2.6. MI_DISP_DeviceDetach	17
2.7. MI_DISP_EnableVideoLayer	18
2.8. MI_DISP_DisableVideoLayer	19
2.9. MI_DISP_SetVideoLayerAttr	20
2.10. MI_DISP_GetVideoLayerAttr	21
2.11. MI_DISP_BindVideoLayer	21
2.12. MI_DISP_UnBindVideoLayer	25
2.13. MI_DISP_SetPlayToleration	26
2.14. MI_DISP_GetPlayToleration	27
2.15. MI_DISP_GetScreenFrame	28
2.16. MI_DISP_ReleaseScreenFrame	28
2.17. MI_DISP_SetVideoLayerAttrBatchBegin	29
2.18. MI_DISP_SetVideoLayerAttrBatchEnd	30
2.19. MI_DISP_EnableInputPort	31
2.20. MI_DISP_DisableInputPort	31
2.21. MI_DISP_SetInputPortAttr	32
2.22. MI_DISP_GetInputPortAttr	33
2.23. MI_DISP_SetInputPortDispPos	34
2.24. MI_DISP_GetInputPortDispPos	34
2.25. MI_DISP_PauseInputPort	35
2.26. MI_DISP_ResumeInputPort	36
2.27. MI_DISP_StepInputPort	37
2.28. MI_DISP_ShowInputPort	38
2.29. MI_DISP_HideInputPort	39

2.30. MI_DISP_SetInputPortSyncMode.....	39
2.31. MI_DISP_QueryInputPortStat.....	40
2.32. MI_DISP_SetZoomInWindow.....	41
2.33. MI_DISP_GetVgaParam.....	42
2.34. MI_DISP_SetVgaParam.....	42
2.35. MI_DISP_GetHdmiParam.....	43
2.36. MI_DISP_SetHdmiParam.....	44
2.37. MI_DISP_GetLcdParam.....	44
2.38. MI_DISP_SetLcdParam.....	45
2.39. MI_DISP_GetCvbsParam.....	46
2.40. MI_DISP_SetCvbsParam.....	47
2.41. MI_DISP_DeviceGetColorTempeture.....	47
2.42. MI_DISP_DeviceSetColorTempeture.....	48
2.43. MI_DISP_DeviceSetGammaParam.....	49
2.44. MI_DISP_SetVideoLayerRotateMode.....	50
2.45. MI_DISP_ClearInputPortBuffer.....	50
2.46. MI_DISP_InitDev.....	51
2.47. MI_DISP_DeInitDev.....	52
2.48. MI_DISP_SetWBCSource.....	53
2.49. MI_DISP_GetWBCSource.....	54
2.50. MI_DISP_SetWBCAttr.....	55
2.51. MI_DISP_GetWBCAttr.....	56
2.52. MI_DISP_EnableWBC.....	56
2.53. MI_DISP_DisableWBC.....	57
3. DISP 数据类型.....	59
3.1. MI_DISP_DEV.....	60
3.2. MI_DISP_LAYER.....	60
3.3. MI_DISP_INPUTPORT.....	60
3.4. MI_DISP_Interface_e.....	61
3.5. MI_DISP_IntfSync_e.....	62
3.6. MI_DISP_SyncInfo_t.....	63
3.7. MI_DISP_PubAttr_t.....	64
3.8. MI_DISP_DevAttachMode_e.....	65
3.9. MI_DISP_DevAttachAttr_t.....	66
3.10. MI_DISP_Csc_t.....	66
3.11. MI_DISP_CscMatrix_e.....	67
3.12. MI_DISP_VgaParam_t.....	68
3.13. MI_DISP_HdmiParam_t.....	68
3.14. MI_DISP_LcdParam_t.....	69
3.15. MI_DISP_CvbsParam_t.....	69
3.16. MI_DISP_ColorTemperature_t.....	70
3.17. MI_DISP_GammaParam_t.....	70
3.18. MI_DISP_VideoLayerSize_t.....	71
3.19. MI_DISP_VideoLayerAttr_t.....	72

3.20. MI_DISP_RotateMode_e.....	72
3.21. MI_DISP_RotateConfig_t.....	73
3.22. MI_DISP_VidWinRect_t.....	73
3.23. MI_DISP_InputPortAttr_t.....	74
3.24. MI_DISP_Position_t.....	75
3.25. MI_DISP_SyncMode_e.....	76
3.26. MI_DISP_InputPortStatus_e.....	76
3.27. MI_DISP_QueryChannelStatus_t.....	77
3.28. MI_DISP_VideoFrame_t.....	77
3.29. MI_DISP_InitParam_t.....	78
3.30. MI_DISP_WBC.....	79
3.31. MI_DISP_WBC_SourceType_e.....	79
3.32. MI_DISP_WBC_Source_t.....	80
3.33. MI_DISP_WBC_TargetSize_t.....	80
3.34. MI_DISP_WBC_Attr_t.....	81
4. DISP 错误码.....	82
5. PROCFS 介绍.....	84
5.1. DISP cat.....	84
5.2. DISP echo.....	86
5.3. WBC cat.....	89
5.4. WBC echo.....	90

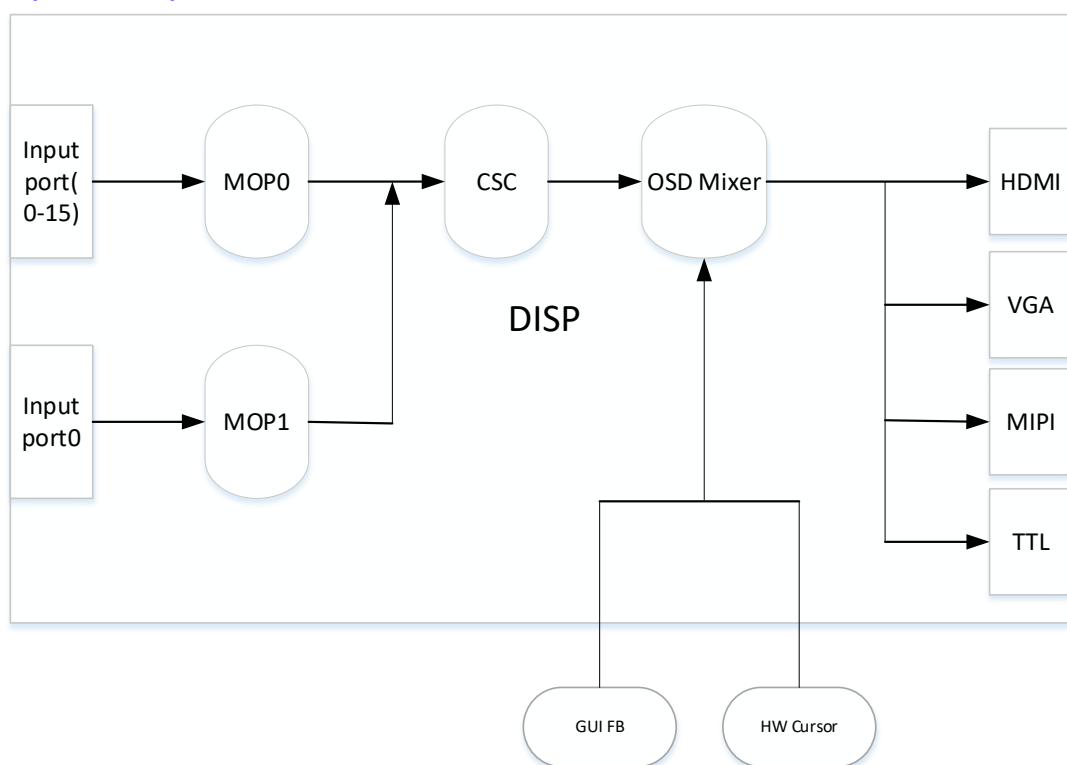
1. 概述

1.1. 模块说明

DISP 是一个视频显示单元，主要功能是对前端输出的图像做硬件拼图，并对硬件拼图后的图像进行颜色空间转换，最终通过 HDMI/VGA/MIPI/TTL 等接口输出到显示器或 LCD。

1.2. 流程框图

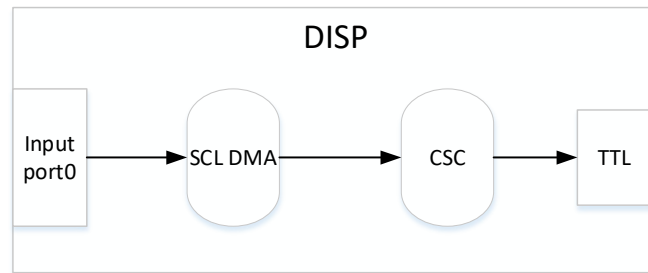
1.2.1. Taiyaki/Takoyaki



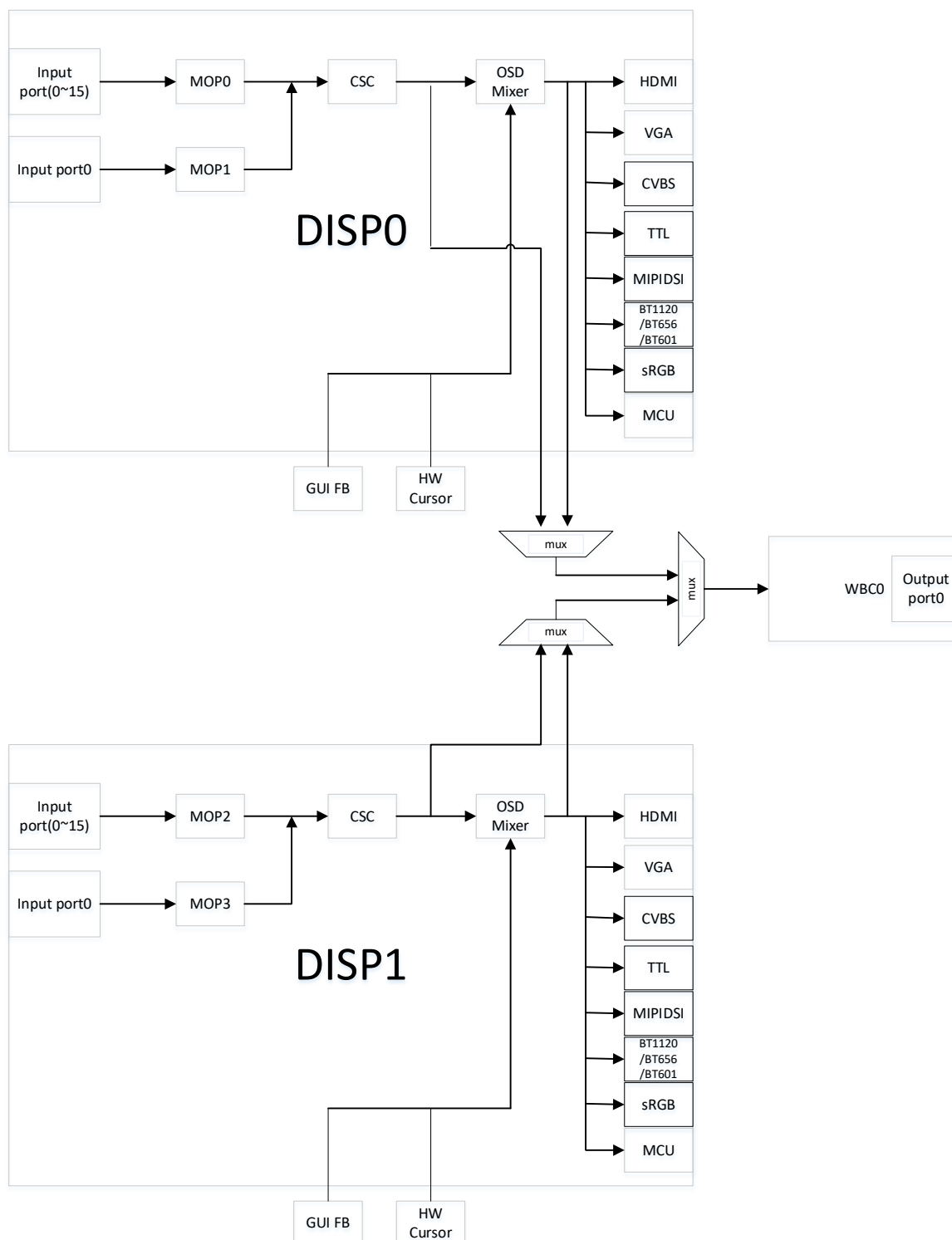
※ 注意

1. HDMI 和 VGA 接口可以同时输出，MIPI、TTL 接口只能单独输出。
2. MOP1 的输出画面会叠加到 MOP0 上，实现 PIP 的功能，即 MOP1 的显示优先级高于 MOP0。
3. MOP0 上面的 16 个 Input Port 的显示位置不能互相叠加。

1.2.2. Pretzel/Pudding



1.2.3. Tiramisu

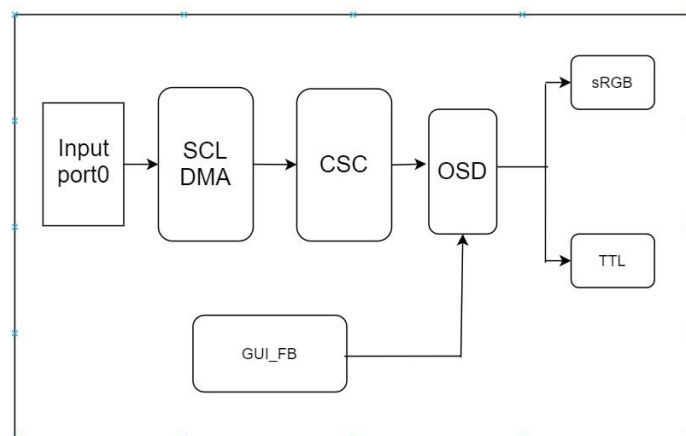


※ 注意

1. 对于每一个 device, HDMI 和 VGA 接口可以同时输出, CVBS、MIPI、TTL、BT1120、BT656、BT601、sRGB、MCU 接口只能单独输出。
2. MOP1 的输出画面会叠加到 MOP0 上, 实现 PIP 的功能, 即 MOP1 的显示优先级高于 MOP0。
3. MOP0 上面的 16 个 Input port 的显示位置不能互相叠加。

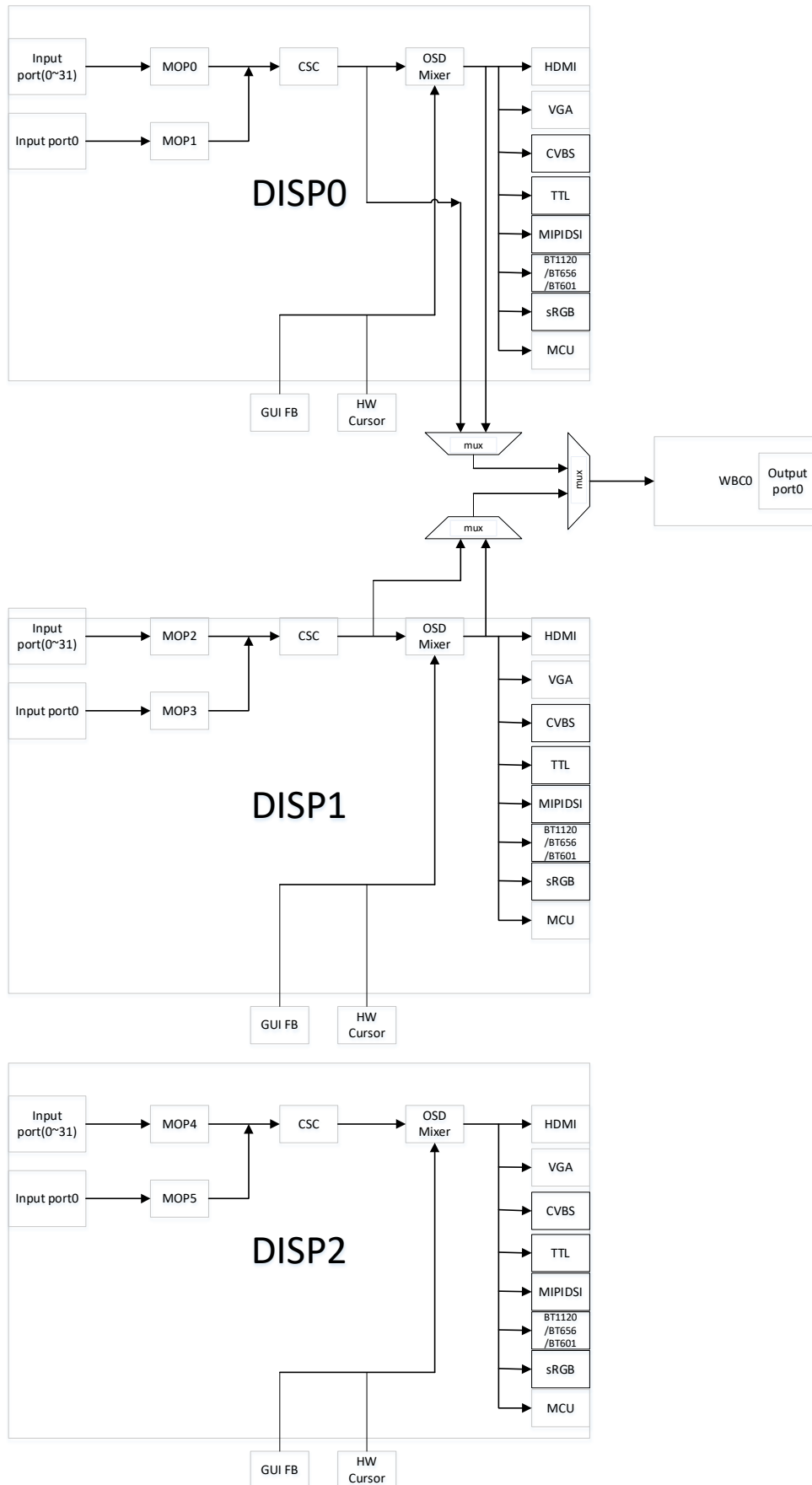
4. MOP2 和 MOP3 的使用方式同 MOP0 和 MOP1。
5. DISP0 和 DISP1 的输出可以通过 WBC 回写到内存。

1.2.4. Ikayaki



※ 注意
TTL 和 sRGB 不能同时输出

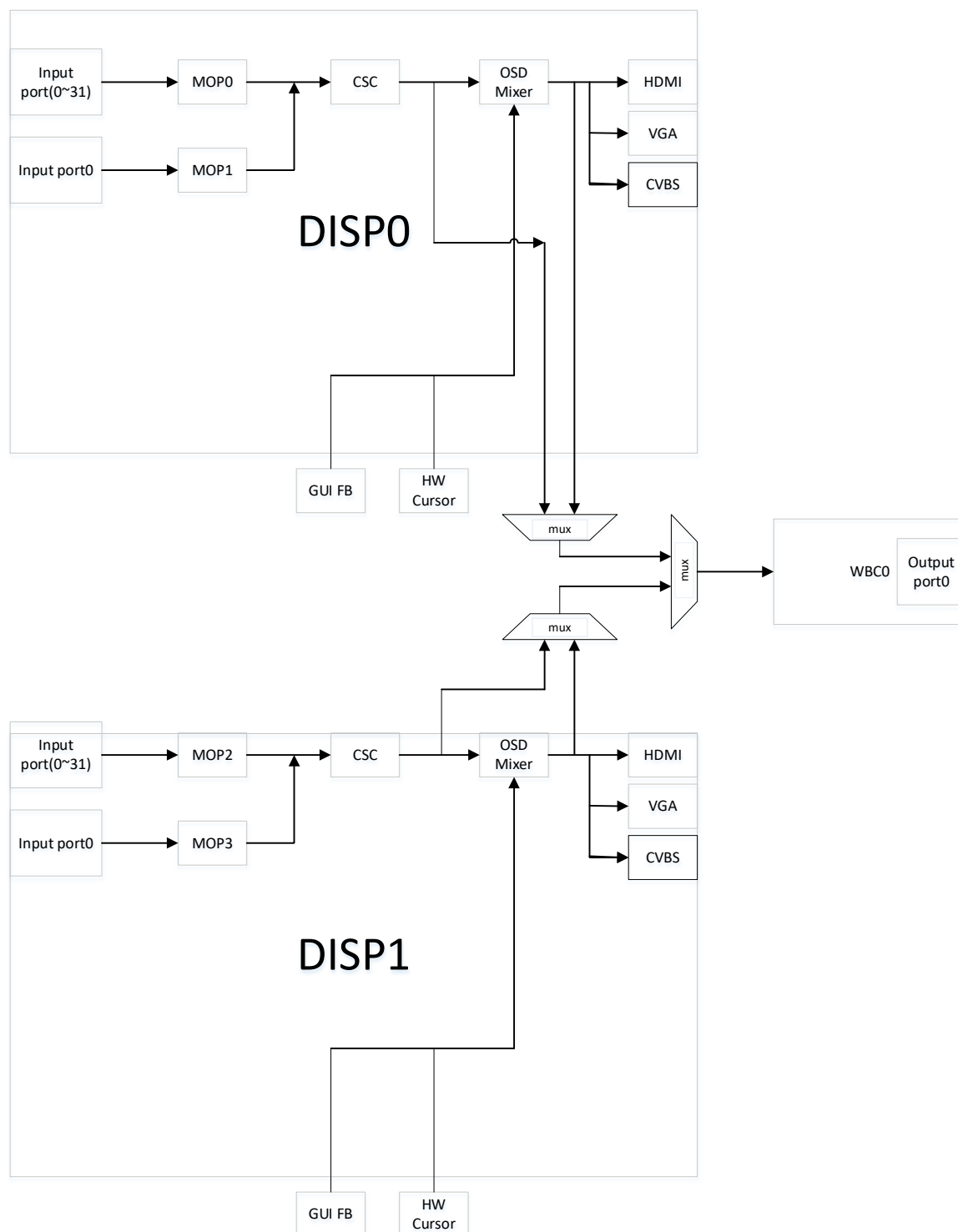
1.2.5. Muffin



※ 注意

1. 对于每一个 device，HDMI 和 VGA 接口可以同时输出，
CVBS、MIPI 、TTL 、BT1120 、BT656 、BT601 、sRGB、MCU 接口只能单独输出。
2. MOP1 的输出画面会叠加到 MOP0 上，实现 PIP 的功能，即 MOP1 的显示优先级高于 MOP0。
3. MOP0 上面的 32 个 Input port 的显示位置不能互相叠加。
4. MOP2、MOP3 和 MOP4、MOP5 的使用方式同 MOP0 和 MOP1。
5. DISP0 和 DISP1 的输出可以通过 WBC 回写到内存。

1.2.6. Mochi

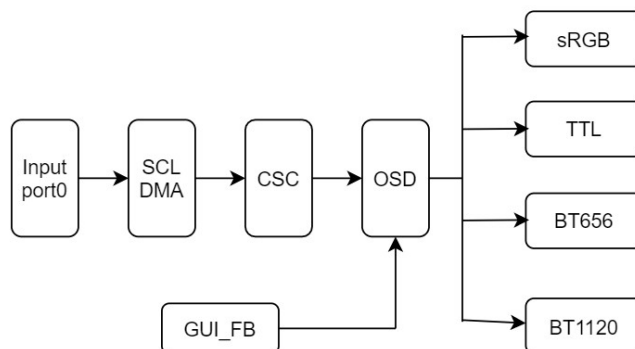


※ 注意

1. 对于每一个 device，HDMI 和 VGA 接口可以同时输出，CVBS 接口只能单独输出。
2. MOP1 的输出画面会叠加到 MOP0 上，实现 PIP 的功能，即 MOP1 的显示优先级高于 MOP0。
3. MOP0 上面的 32 个 Input port 的显示位置不能互相叠加。
4. MOP2、MOP3 的使用方式同 MOP0 和 MOP1。

5. DISP0 和 DISP1 的输出可以通过 WBC 回写到内存。

1.2.7. Maruko



※ 注意

sRGB,TTL,BT656 和 BT1120 不能同时输出，同一时间只能输出其中一种

※ 注意

下表是不同芯片系列输出接口的差异

输出接口	HDMI	VGA	MIPI DSI	TTL	CVBS	sRGB
芯片系列						
Pretzel	不支持	不支持	不支持	支持	不支持	不支持
Macaron	不支持	不支持	不支持	不支持	不支持	不支持
Taiyaki	支持	支持	不支持	不支持	不支持	不支持
Takoyaki	不支持	不支持	支持	支持	不支持	不支持
Pudding	不支持	不支持	不支持	支持	不支持	不支持
Ispahan	不支持	不支持	不支持	不支持	不支持	不支持
MSR930	支持	支持	支持	支持	不支持	不支持
Tiramisu	支持	支持	支持	支持	支持	支持
Ikayaki	不支持	不支持	不支持	支持	不支持	支持
Muffin	支持	支持	支持	支持	支持	支持
Mochi	支持	支持	不支持	不支持	支持	不支持
Maruko	不支持	不支持	不支持	支持	不支持	支持

1.3. 关键字说明

● DEV

显示设备。对应 [1.2 流程框图](#) 的 DISP0/DISP1 等

● MOP

读取内存图像数据并做拼图处理的硬件单元

- **LAYER**
视频层。MOP 硬件抽象层。
- **CSC**
颜色空间转换单元
- **OSD Mixer**
视频层和 UI 叠加器件
- **GUI FB/HW Cursor**
UI 层
- **PIP**
画中画
- **WBC**
视频回写设备。捕获视频层或设备级的视频数据，可用于多 dev 的同源显示。

2. API 参考

表 2-1 API 参考

API 名	功能
MI_DISP_Enable	启用视频输出设备
MI_DISP_Disable	禁用视频输出设备
MI_DISP_SetPubAttr	配置视频输出设备的公共属性
MI_DISP_GetPubAttr	获取视频输出设备的公共属性
MI_DISP_DeviceAttach	使能视频同源输出
MI_DISP_DeviceDetach	禁用视频同源输出
MI_DISP_EnableVideoLayer	使能视频层
MI_DISP_DisableVideoLayer	禁用视频层
MI_DISP_SetVideoLayerAttr	设定视频层属性
MI_DISP_GetVideoLayerAttr	获取视频层属性
MI_DISP_BindVideoLayer	绑定视频层到指定设备
MI_DISP_UnBindVideoLayer	解绑视频层与指定设备
MI_DISP_SetPlayToleration	设置播放容忍度
MI_DISP_GetPlayToleration	获取播放容忍度
MI_DISP_GetScreenFrame	获取屏幕输出图像数据
MI_DISP_ReleaseScreenFrame	释放屏幕输出图像数据
MI_DISP_SetVideoLayerAttrBatchBegin	设置视频层上输入端口相关操作开始
MI_DISP_SetVideoLayerAttrBatchEnd	设置视频层上输入端口相关操作结束
MI_DISP_EnableInputPort	启用指定的视频输入端口
MI_DISP_DisableInputPort	禁用指定的视频输入端口
MI_DISP_SetInputPortAttr	配置指定视频输入端口的属性
MI_DISP_GetInputPortAttr	获取指定视频输入端口的属性
MI_DISP_SetInputPortDispPos	设置指定视频输入端口的显示位置
MI_DISP_GetInputPortDispPos	获取指定视频输入端口的显示位置
MI_DISP_PauseInputPort	暂停指定的视频输入端口
MI_DISP_ResumeInputPort	恢复指定的视频输入端口
MI_DISP_StepInputPort	单帧播放指定的视频输入端口
MI_DISP_ShowInputPort	显示指定的视频输入端口

API 名	功能
MI_DISP_HideInputPort	隐藏指定的视频输入端口
MI_DISP_SetInputPortSyncMode	设定指定的视频输入端口同步模式
MI_DISP_QueryInputPortStat	查询视频输入端口状态
MI_DISP_SetZoomInWindow	裁剪视频输入端口
MI_DISP_GetVgaParam	获取 VGA 输出设备参数
MI_DISP_SetVgaParam	设定 VGA 输出设备参数
MI_DISP_GetHdmiParam	获取 HDMI 输出设备参数
MI_DISP_SetHdmiParam	设定 HDMI 输出设备参数
MI_DISP_GetLcdParam	获取 LCD 输出设备参数
MI_DISP_SetLcdParam	设定 LCD 输出设备参数
MI_DISP_GetCvbsParam	获取CVBS输出设备参数
MI_DISP_SetCvbsParam	设定CVBS输出设备参数
MI_DISP_DeviceGetColorTempature	获取当前输出图像的色温参数
MI_DISP_DeviceSetColorTempature	设定当前输出图像的色温参数
MI_DISP_DeviceSetGammaParam	设定当前输出图像的 Gamma
MI_DISP_SetVideoLayerRotateMode	设定当前输出图像的旋转角度
MI_DISP_ClearInputPortBuffer	清除当前显示窗口内容
MI_DISP_InitDev	初始化 disp 设备
MI_DISP_DeInitDev	反初始化 disp 设备
MI_DISP_SetWBCSource	设置回写设备的回写源
MI_DISP_GetWBCSource	获取回写设备的回写源
MI_DISP_SetWBCAttr	设置回写设备属性
MI_DISP_GetWBCAttr	获取回写设备属性
MI_DISP_EnableWBC	使能回写设备
MI_DISP_DisableWBC	禁用回写设备

2.1. MI_DISP_Enable

➤ 功能

启用视频输出设备。

➤ 语法

MI_S32 MI_DISP_Enable ([MI_DISP_DEV](#) DispDev);

➤ 形参

参数名称	描述	输入/输出
DispDev	视频输出设备号。	输入

➤ 返回值

返回值 {
 0 成功。
 非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

※ 注意

- 由于系统没有初始化设备为使能状态，所以在使用视频输出功能前必须先进行设备使能操作。
- 在调用设备使能前，必须对设备公共属性进行配置，否则返回设备未配置错误。

➤ 举例

```
MI_U32 DispDev = 0;
MI_U32 DispLayer = 0;
MI_U32 DispInport = 0;
MI_DISP_PubAttr_t stPubAttr;
MI_DISP_VideoLayerAttr_t stLayerAttr;
MI_DISP_InputPortAttr_t stInputPortAttr;
MI_DISP_VidWinRect_t stWinRect;
MI_DISP_RotateConfig_t stRotateConfig;

memset(&stPubAttr,0,sizeof(MI_DISP_PubAttr_t));
memset(&stLayerAttr,0,sizeof(MI_DISP_VideoLayerAttr_t));
memset(&stInputPortAttr,0,sizeof(MI_DISP_InputPortAttr_t));
memset(&stWinRect, 0, sizeof(MI_DISP_VidWinRect_t));
memset(&stRotateConfig, 0, sizeof(MI_DISP_RotateConfig_t));

stPubAttr.eIntfSync = E_MI_DISP_OUTPUT_1080P60;
stPubAttr.eIntfType = E_MI_DISP_INTF_HDMI;
MI_DISP_SetPubAttr(DispDev, &stPubAttr);
stPubAttr.eIntfType = E_MI_DISP_INTF_VGA;
MI_DISP_SetPubAttr(DispDev, &stPubAttr); // Non-essential steps
MI_DISP_Enable(DispDev);
```

```

//HDMI related init
//Necessary steps. Not listed here

stLayerAttr.stVidLayerSize.u16Width = 1920;
stLayerAttr.stVidLayerSize.u16Height = 1080;
stLayerAttr.stVidLayerDispWin.u16X = 0;
stLayerAttr.stVidLayerDispWin.u16Y = 0;
stLayerAttr.stVidLayerDispWin.u16Width = 1920;
stLayerAttr.stVidLayerDispWin.u16Height = 1080;
stRotateConfig.eRotateMode = E_MI_DISP_ROTATE_NONE;
MI_DISP_BindVideoLayer(DispLayer, DispDev);
MI_DISP_SetVideoLayerAttr(DispLayer, &stLayerAttr);
MI_DISP_EnableVideoLayer(DispLayer);
MI_DISP_SetVideoLayerRotateMode(DispLayer, &stRotateConfig); // Non-essential steps

stInputPortAttr.u16SrcWidth = 1920;
stInputPortAttr.u16SrcHeight = 1080;
stInputPortAttr.stDispWin.u16X = 0;
stInputPortAttr.stDispWin.u16Y = 0;
stInputPortAttr.stDispWin.u16Width = 1920;
stInputPortAttr.stDispWin.u16Height = 1080;
stWinRect.u16X = 0;
stWinRect.u16Y = 0;
stWinRect.u16Width = 1920;
stWinRect.u16Height = 1080;
MI_DISP_SetInputPortAttr(DispLayer, DispInport, &stInputPortAttr);
MI_DISP_SetZoomInWindow(DispLayer, DispInport, &stWinRect); // Non-essential steps
MI_DISP_SetVideoLayerAttrBatchBegin(DispLayer); // Non-essential steps
MI_DISP_EnableInputPort(DispLayer, DispInport);
MI_DISP_SetVideoLayerAttrBatchEnd(DispLayer); // Non-essential steps

//exit flow
MI_DISP_SetVideoLayerAttrBatchBegin(DispLayer); // Non-essential steps
MI_DISP_DisableInputPort(DispLayer, DispInport);
MI_DISP_SetVideoLayerAttrBatchEnd(DispLayer); // Non-essential steps
MI_DISP_DisableVideoLayer(DispLayer);
MI_DISP_UnBindVideoLayer(DispLayer, DispDev);
MI_DISP_Disable(DispDev);

```

➤ 相关主题

[MI_DISP_Disable](#)

2.2. MI_DISP_Disable

➤ 功能

禁用视频输出设备。

➤ 语法

```
MI_S32 MI_DISP_Disable(MI\_DISP\_DEV DispDev);
```

➤ 形参

参数名称	描述	输入/输出
DispDev	视频输出设备号。	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

※ 注意

无

➤ 举例

无

➤ 相关主题

[MI_DISP_Enable](#)

2.3. MI_DISP_SetPubAttr

➤ 功能

配置视频输出设备的公共属性。

➤ 语法

MI_S32 MI_DISP_SetPubAttr([MI_DISP_DEV](#) DispDev, [MI_DISP_PubAttr_t](#) *pstPubAttr);

➤ 形参

参数名称	描述	输入/输出
DispDev	输出设备号。	输入
pstPubAttr	输出设备公共属性结构体指针。	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

※ 注意

无

➤ 举例

无

➤ 相关主题

[MI_DISP_GetPubAttr](#)

2.4. MI_DISP_GetPubAttr

➤ 功能

获取视频输出设备的公共属性。

➤ 语法

MI_S32 MI_DISP_GetPubAttr ([MI_DISP_DEV](#) DispDev, [MI_DISP_PubAttr_t](#) *pstPubAttr);

➤ 形参

参数名称	描述	输入/输出
DispDev	输出设备号。	输入
pstPubAttr	输出设备公共属性结构体指针。	输出

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

※ 注意

无

➤ 举例

无

➤ 相关主题

[MI_DISP_SetPubAttr](#)

2.5. MI_DISP_DeviceAttach

➤ 功能

使能视频同源输出，可以实现不同的输出接口同步显示相同画面。

➤ 语法

MI_S32 MI_DISP_DeviceAttach([MI_DISP_DEV](#) DispSrcDev, [MI_DISP_DEV](#) DispDstDev, const [MI_DISP_DevAttachAttr_t](#) *pstAttachAttr);

➤ 形参

参数名称	描述	输入/输出
DispSrcDev	输出设备号（视频源输入设备）。	输入
DispDstDev	输出设备号（同源输出设备）。	输入
pstAttachAttr	视频同源输出属性	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

※ 注意

- 适用于有两个可以同源视频输出设备的芯片。
- 目前只支持 MSR930、MSR650x。

➤ 举例

无

➤ 相关主题

[MI_DISP_DeviceDetach](#)

2.6. MI_DISP_DeviceDetach

➤ 功能

禁用视频同源输出。

➤ 语法

MI_S32 MI_DISP_DeviceDetach([MI_DISP_DEV](#) DispSrcDev, [MI_DISP_DEV](#) DispDstDev);

➤ 形参

参数名称	描述	输入/输出
DispSrcDev	输出设备号（视频源输入设备）。	输入
DispDstDev	输出设备号（同源输出设备）。	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h

- 库文件：libmi_disp.a / libmi_disp.so

※ 注意

- 适用于有两个可以同源视频输出设备的芯片。
- 目前只支持 MSR930、MSR650x。

➤ 举例

无

➤ 相关主题

[MI_DISP_DeviceAttach](#)

2.7. MI_DISP_EnableVideoLayer

➤ 功能

使能视频层

➤ 语法

MI_S32 MI_DISP_EnableVideoLayer ([MI_DISP_LAYER](#) DispLayer);

➤ 形参

参数名称	描述	输入/输出
DispLayer	视频输出视频层号。	输入

➤ 返回值

返回值 {
 0 成功。
 非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

※ 注意

无

➤ 举例

无

➤ 相关主题

[MI_DISP_DisableVideoLayer](#)

2.8. MI_DISP_DisableVideoLayer

➤ 功能

禁用视频层

➤ 语法

MI_S32 MI_DISP_DisableVideoLayer ([MI_DISP_LAYER](#) DispLayer);

➤ 形参

参数名称	描述	输入/输出
DispLayer	视频输出视频层号。	输入

- 返回值

返回值 {

0 成功。

非 0 失败，详情参照[错误码](#)。
- 依赖
 - 头文件：mi_disp.h、mi_disp_datatype.h
 - 库文件：libmi_disp.a / libmi_disp.so
- ※ 注意
 - 禁用视频层前需禁用其上全部的输入端口
- 举例

无
- 相关主题

[MI_DISP_EnableVideoLayer](#)

2.9. MI_DISP_SetVideoLayerAttr

- 功能

设定视频层属性。
- 语法


```
MI_S32 MI_DISP_SetVideoLayerAttr(MI\_DISP\_LAYER DispLayer, MI\_DISP\_VideoLayerAttr\_t *pstLayerAttr);
```
- 形参

参数名称	描述	输入/输出
DispLayer	视频输出视频层号。	输入
pstLayerAttr	视频层属性结构体指针	输入
- 返回值

返回值 {

0 成功。

非 0 失败，详情参照[错误码](#)。
- 依赖
 - 头文件：mi_disp.h、mi_disp_datatype.h
 - 库文件：libmi_disp.a / libmi_disp.so
- ※ 注意

无
- 举例

无

➤ 相关主题

[MI_DISP_GetVideoLayerAttr](#)

2.10. MI_DISP_GetVideoLayerAttr

➤ 功能

获取视频层属性。

➤ 语法

```
MI_S32 MI_DISP_GetVideoLayerAttr(MI\_DISP\_LAYER DispLayer, MI\_DISP\_VideoLayerAttr\_t *pstLayerAttr);
```

➤ 形参

参数名称	描述	输入/输出
DispLayer	视频输出视频层号。	输入
pstLayerAttr	视频层属性结构体指针	输出

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

※ 注意

无

➤ 举例

无

➤ 相关主题

[MI_DISP_SetVideoLayerAttr](#)

2.11. MI_DISP_BindVideoLayer

➤ 功能

绑定视频层到指定设备。

➤ 语法

```
MI_S32 MI_DISP_BindVideoLayer(MI\_DISP\_LAYER DispLayer, MI\_DISP\_DEV DispDev);
```

➤ 形参

参数名称	描述	输入/输出
DispLayer	视频输出视频层号。	输入
DispDev	输出设备号。	输入

➤ 返回值

返回值 {
0 成功。
非 0 失败，详情参照[错误码](#)。

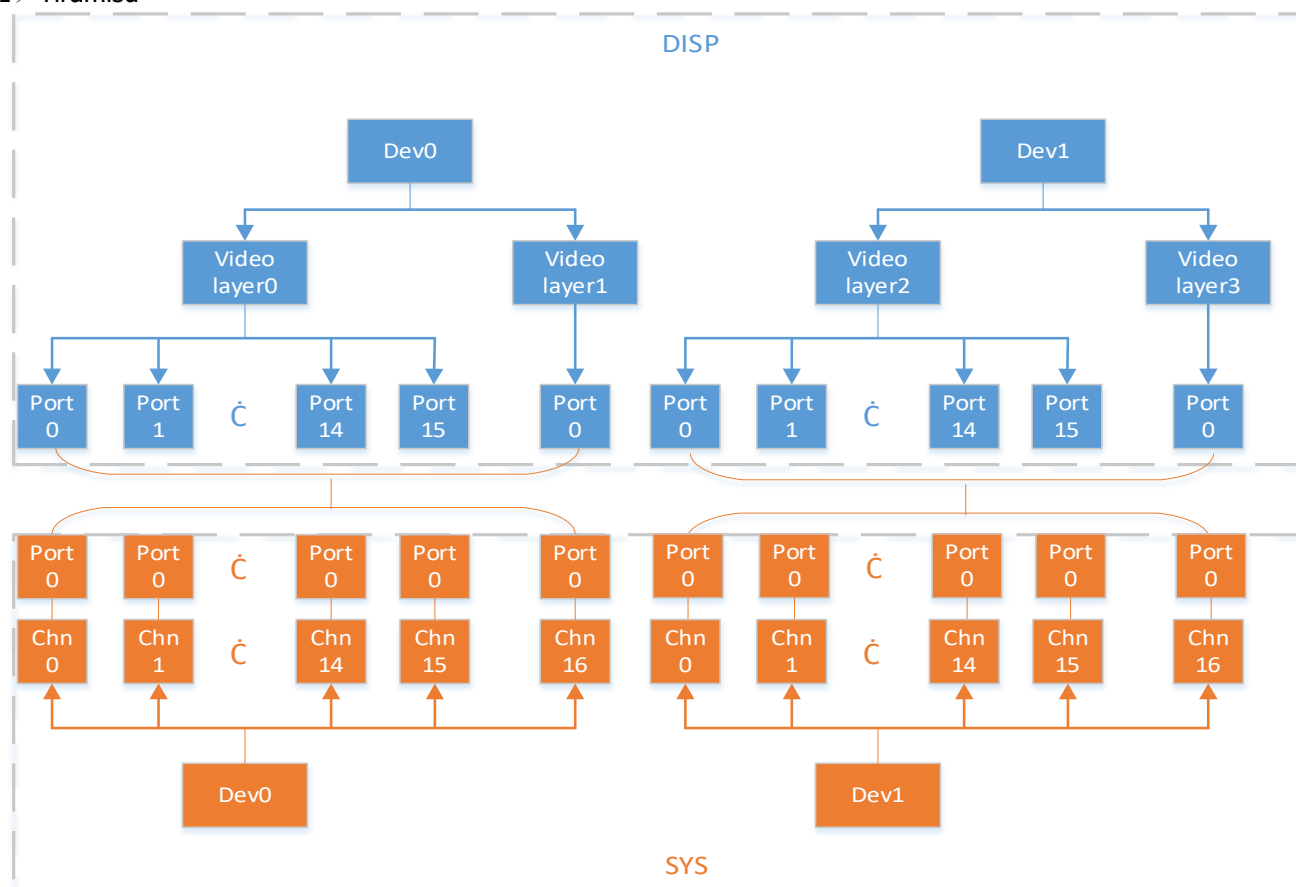
➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

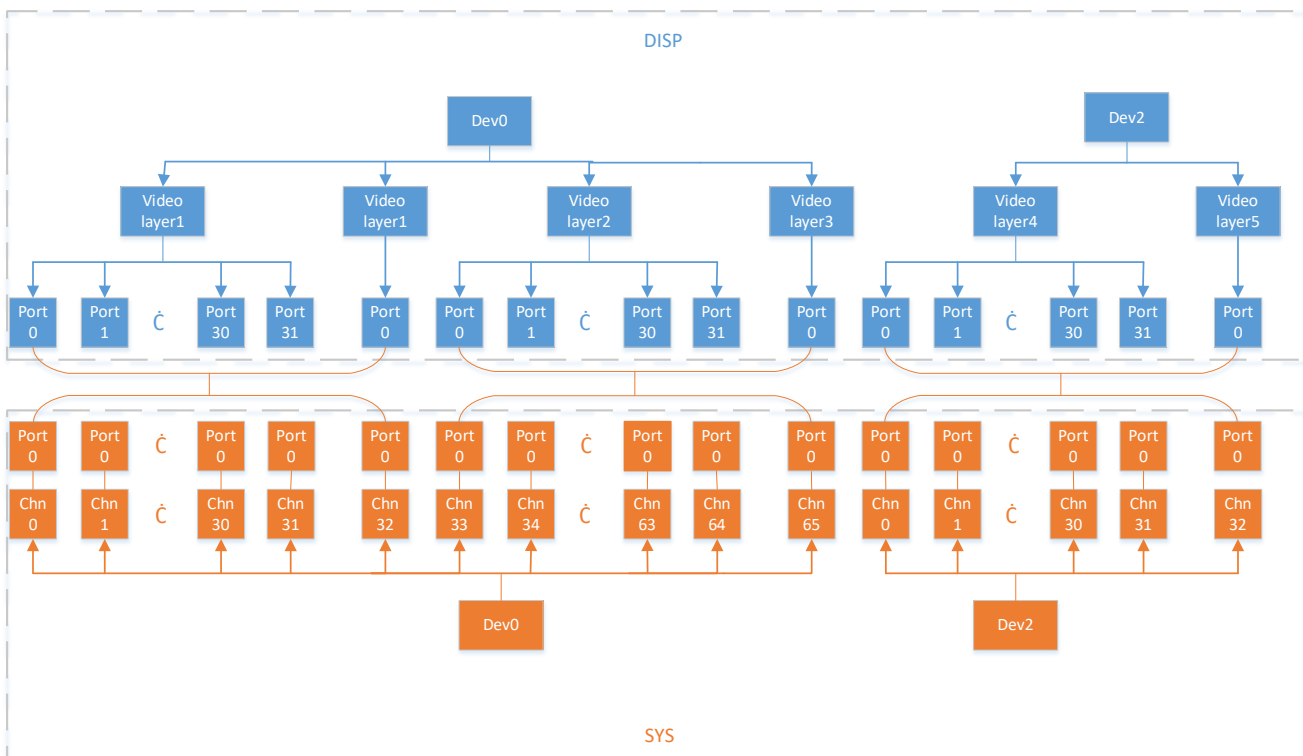
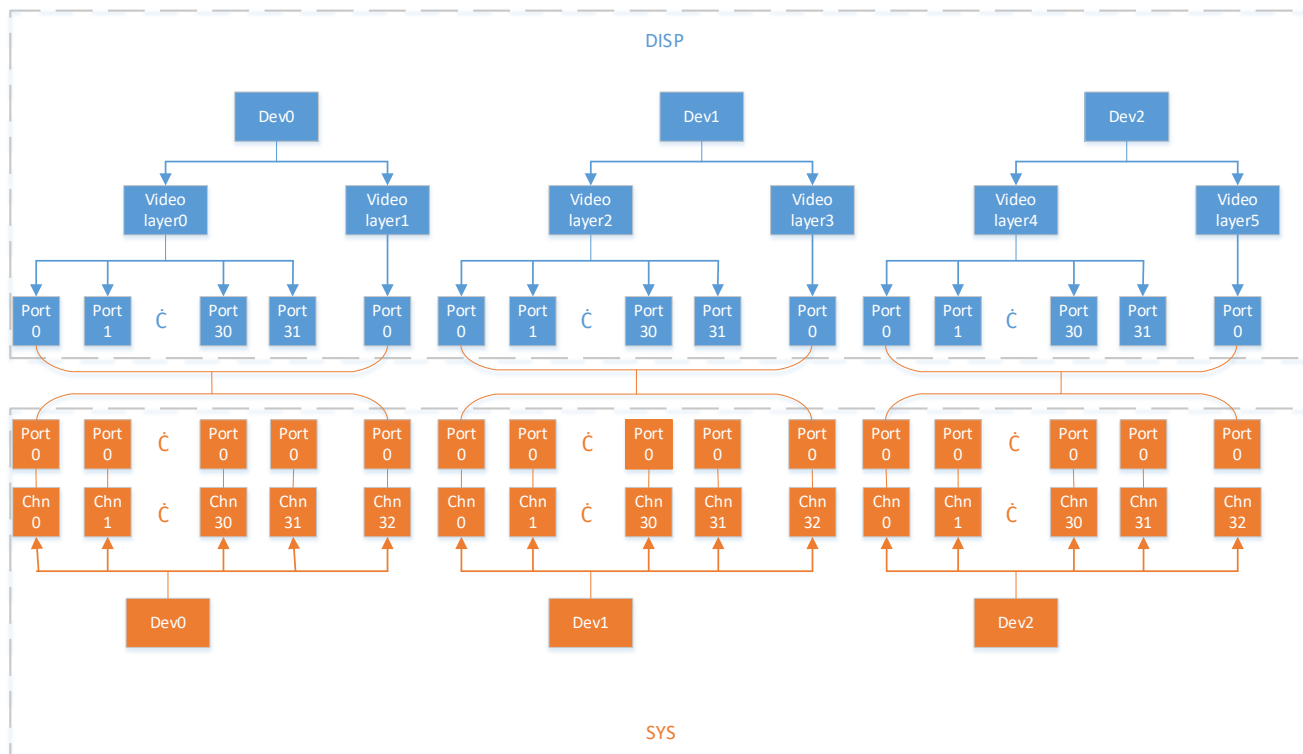
※ 注意

- 调用前保证视频层所有的输入端口处于禁用状态
- disp 和其他模块绑定时，disp 的 device、video layer、input port 和对应的 channel、port 映射关系如下

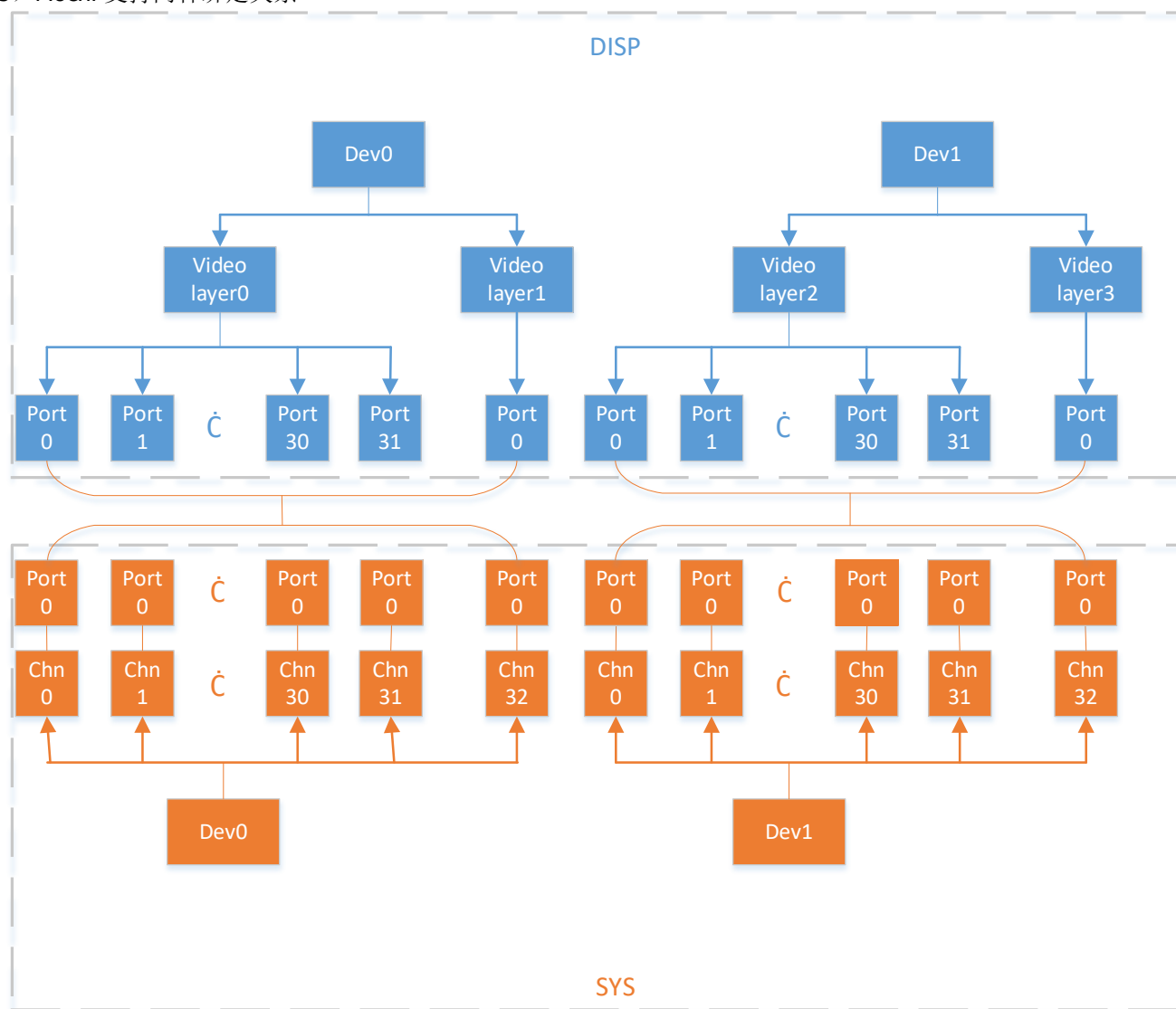
1) Tiramisu

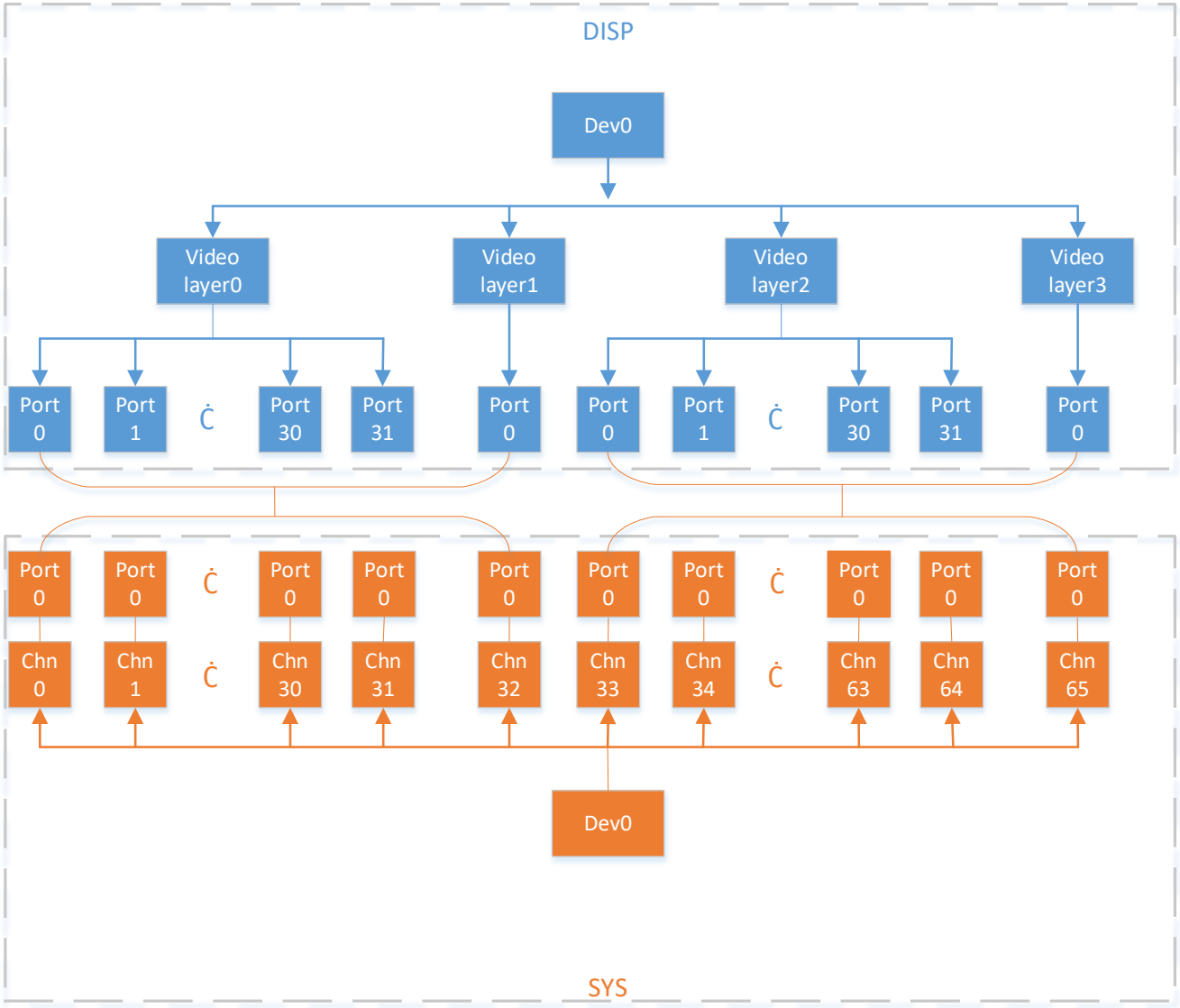


2) Muffin 支持两种绑定关系



3) Mochi 支持两种绑定关系





➤ 举例
无

➤ 相关主题
[MI_DISP_UnBindVideoLayer](#)

2.12. MI_DISP_UnBindVideoLayer

➤ 功能
解绑视频层与指定设备

➤ 语法
MI_S32 MI_DISP_UnBindVideoLayer([MI_DISP_LAYER](#) DispLayer, [MI_DISP_DEV](#) DispDev);

➤ 形参

参数名称	描述	输入/输出
------	----	-------

DispLayer	视频输出视频层号。	输入
DispDev	输出设备号。	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

※ 注意

- 解绑前需禁用其上全部的输入端口。
- 解绑前需禁用视频层。

➤ 举例

无

➤ 相关主题

[MI_DISP_BindVideoLayer](#)

2.13. MI_DISP_SetPlayToleration

➤ 功能

设置播放容忍度。

➤ 语法

MI_S32 MI_DISP_SetPlayToleration([MI_DISP_LAYER](#) DispLayer, MI_U32 u32Toleration);

➤ 形参

参数名称	描述	输入/输出
DispLayer	视频输出视频层号。	输入
u32Toleration	播放容忍度，用来调整帧率控制算法的误差允许范围	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

※ 注意

- 播放容忍度单位为毫秒。
- 调用前请保证视频层已经使能。
- 目前只支持 MSR930、MSR650x。

➤ 举例

无

➤ 相关主题

[MI_DISP_GetPlayToleration](#)

2.14. MI_DISP_GetPlayToleration

➤ 功能

获取播放容忍度。

➤ 语法

MI_S32 MI_DISP_GetPlayToleration ([MI_DISP_LAYER](#) DispLayer, MI_U32 *pu32Toleration);

➤ 形参

参数名称	描述	输入/输出
DispLayer	视频输出视频层号。	输入
pu32Toleration	播放容忍度。	输出

➤ 返回值

返回值 {
 0 成功。
 非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

※ 注意

- 播放容忍度单位为毫秒。
- 调用前请保证视频层已经使能。
- 目前只支持 MSR930、MSR650x。

➤ 举例

无

➤ 相关主题

[MI_DISP_SetPlayToleration](#)

2.15. MI_DISP_GetScreenFrame

➤ 功能

获取输出屏幕图像数据。

➤ 语法

```
MI_S32 MI_DISP_GetScreenFrame(MI\_DISP\_LAYER DispLayer, MI\_DISP\_VideoFrame\_t
*pstVFrame, MI_S32 s32MilliSec);
```

➤ 形参

参数名称	描述	输入/输出
DispLayer	视频输出视频层号。	输入
pstVFrame	获取的输出屏幕图像数据信息结构体指针。	输出
s32MilliSec	超时参数 s32MilliSec 设为-1 时，为阻塞接口；0 时为非阻塞接口；大于 0 时为超时等待时间。超时时间的单位为毫秒（ms）。	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

※ 注意

- 目前只支持 MSR930、MSR650x。
- 调用前请保证视频层已经使能。

➤ 举例

无

➤ 相关主题

[MI_DISP_ReleaseScreenFrame](#)

2.16. MI_DISP_ReleaseScreenFrame

➤ 功能

释放输出屏幕图像数据。

➤ 语法

```
MI_S32 MI_DISP_ReleaseScreenFrame (MI\_DISP\_LAYER DispLayer,
MI\_DISP\_VideoFrame\_t *pstVFrame);
```

➤ 形参

参数名称	描述	输入/输出
DispLayer	视频输出视频层号。	输入
pstVFrame	获取的输出屏幕图像数据信息结构体指针。	输入

- 返回值
- 返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

- 依赖
- 头文件：mi_disp.h、mi_disp_datatype.h
 - 库文件：libmi_disp.a / libmi_disp.so

- ※ 注意
- 目前只支持 MSR930、MSR650x。
 - 调用前请保证视频层已经使能。

- 举例
- 无

- 相关主题
- [MI_DISP_GetScreenFrame](#)

2.17. MI_DISP_SetVideoLayerAttrBatchBegin

- 功能
- 设置视频层上输入端口相关操作开始。
- 语法
- MI_S32 MI_DISP_SetVideoLayerAttrBatchBegin ([MI_DISP_LAYER](#) DispLayer);

- 形参

参数名称	描述	输入/输出
DispLayer	视频输出视频层号。	输入

- 返回值
- 返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

- 依赖
- 头文件：mi_disp.h、mi_disp_datatype.h
 - 库文件：libmi_disp.a / libmi_disp.so

- ※ 注意
- 调用前请保证视频层已经使能。
 - [MI_DISP_SetVideoLayerAttrBatchBegin](#) 和 [MI_DISP_SetVideoLayerAttrBatchEnd](#) 接口要配对使用，否则 [MI_DISP_SetVideoLayerAttrBatchBegin](#) 后设置视频层上输入端口相关操作不会生效。
 - 此接口支持以下接口的批处理：[MI_DISP_EnableInputPort](#)、[MI_DISP_DisableInputPort](#)、[MI_DISP_HideInputPort](#)、

- [MI_DISP_ShowInputPort](#)
- 对于不支持批处理的接口会立即生效。
- 批处理中输入端口操作都是在 [MI_DISP_SetVideoLayerAttrBatchEnd](#) 调用之后生效。
例如批处理禁用输入端口 0~15，接口顺序为
[MI_DISP_SetVideoLayerAttrBatchBegin](#)、[MI_DISP_DisableInputPort](#)（0~15）、[MI_DISP_SetVideoLayerAttrBatchEnd](#)。
- 目前支持 Muffin、Mochi。

➤ 举例

无

➤ 相关主题

[MI_DISP_SetVideoLayerAttrBatchEnd](#)

2.18. MI_DISP_SetVideoLayerAttrBatchEnd

➤ 功能

设置视频层上输入端口相关操作结束。

➤ 语法

MI_S32 MI_DISP_SetVideoLayerAttrBatchEnd ([MI_DISP_LAYER](#) DispLayer);

➤ 形参

参数名称	描述	输入/输出
DispLayer	视频输出视频层号。	输入

➤ 返回值

返回值 {
0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

※ 注意

- 调用前请保证 [MI_DISP_SetVideoLayerAttrBatchBegin](#) 已被调用。
- 调用前请保证视频层已经使能。
- 目前支持 Muffin、Mochi。

➤ 举例

无

➤ 相关主题

[MI_DISP_SetVideoLayerAttrBatchBegin](#)

2.19. MI_DISP_EnableInputPort

➤ 功能

启用指定的视频输入端口。

➤ 语法

```
MI_S32 MI_DISP_EnableInputPort (MI_DISP_LAYER DispLayer, MI_DISP_INPUTPORT
LayerInputPort);
```

➤ 形参

参数名称	描述	输入/输出
DispLayer	视频输出视频层号。	输入
LayerInputPort	视频输入端口号。	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

※ 注意

- 调用前必须使能视频层
- 调用前必须绑定视频层

➤ 举例

无

➤ 相关主题

[MI_DISP_DisableInputPort](#)

2.20. MI_DISP_DisableInputPort

➤ 功能

禁用指定的视频输入端口。

➤ 语法

```
MI_S32 MI_DISP_DisableInputPort (MI_DISP_LAYER DispLayer, MI_DISP_INPUTPORT
LayerInputPort);
```

➤ 形参

参数名称	描述	输入/输出
DispLayer	视频输出视频层号。	输入
LayerInputPort	视频输入端口号。	输入

- 返回值

返回值 {

0 成功。

非 0 失败，详情参照[错误码](#)。
- 依赖
 - 头文件：mi_disp.h、mi_disp_datatype.h
 - 库文件：libmi_disp.a / libmi_disp.so
- ※ 注意

调用前必须使能视频层。
- 举例

无
- 相关主题

[MI_DISP_EnableInputPort](#)

2.21. MI_DISP_SetInputPortAttr

- 功能

配置指定视频输入端口的属性。
- 语法


```
MI_S32 MI_DISP_SetInputPortAttr(MI\_DISP\_LAYER DispLayer, MI\_DISP\_INPUTPORT LayerInputPort, MI\_DISP\_InputPortAttr\_t* pstInputPortAttr);
```
- 形参

参数名称	描述	输入/输出
DispLayer	视频输出视频层号。	输入
LayerInputPort	视频输入端口号。	输入
pstInputPortAttr	视频输入端口属性指针	输入
- 返回值

返回值 {

0 成功。

非 0 失败，详情参照[错误码](#)。
- 依赖
 - 头文件：mi_disp.h、mi_disp_datatype.h
 - 库文件：libmi_disp.a / libmi_disp.so
- ※ 注意
 - 调用前必须使能视频层
 - 调用前必须绑定视频层

➤ 举例

无

➤ 相关主题

[MI_DISP_GetInputPortAttr](#)

2.22. MI_DISP_GetInputPortAttr

➤ 功能

获取指定视频输入端口的属性。

➤ 语法

```
MI_S32 MI_DISP_GetInputPortAttr
(MI_DISP_LAYER DispLayer, MI_DISP_INPUTPORT LayerInputPort,
MI_DISP_InputPortAttr_t *pstInputPortAttr);
```

➤ 形参

参数名称	描述	输入/输出
DispLayer	视频输出视频层号。	输入
LayerInputPort	视频输入端口号。	输入
pstInputPortAttr	视频输入端口属性指针	输出

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

※ 注意

- 调用前必须使能视频层
- 调用前必须绑定视频层

➤ 举例

无

➤ 相关主题

[MI_DISP_SetInputPortAttr](#)

2.23. MI_DISP_SetInputPortDispPos

➤ 功能

设置指定视频输入端口的显示位置。

➤ 语法

MI_S32 MI_DISP_SetInputPortDispPos([MI_DISP_LAYER](#) DispLayer, [MI_DISP_INPUTPORT](#) LayerInputPort, const [MI_DISP_Position_t](#) *pstDispPos);

➤ 形参

参数名称	描述	输入/输出
DispLayer	视频输出视频层号。	输入
LayerInputPort	视频输入端口号。	输入
pstDispPos	输入端口位置指针	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

※ 注意

调用前必须使能视频层。

➤ 举例

无

➤ 相关主题

[MI_DISP_GetInputPortDispPos](#)[MI_DISP_GetChannelDispPos](#)

2.24. MI_DISP_GetInputPortDispPos

➤ 功能

获取指定视频输入端口的显示位置。

➤ 语法

MI_S32 MI_DISP_GetInputPortDispPos
([MI_DISP_LAYER](#) DispLayer, [MI_DISP_INPUTPORT](#) LayerInputPort, [MI_DISP_Position_t](#) *pstDispPos);

➤ 形参

参数名称	描述	输入/输出
DispLayer	视频输出视频层号。	输入
LayerInputPort	视频输入端口号。	输入
pstDispPos	输入端口位置指针	输出

➤ 返回值

{ 0 成功。

返回值

非 0 失败，详情参照[错误码](#)。

- 依赖
 - 头文件：mi_disp.h、mi_disp_datatype.h
 - 库文件：libmi_disp.a / libmi_disp.so

※ 注意
调用前必须使能视频层。

➤ 举例
无

➤ 相关主题
[MI_DISP_SetInputPortDispPos](#)

2.25. MI_DISP_PauseInputPort

➤ 功能
暂停指定的视频输入端口。

➤ 语法
MI_S32 MI_DISP_PauseInputPort ([MI_DISP_LAYER](#) DispLayer, [MI_DISP_INPUTPORT](#) LayerInputPort);

➤ 形参

参数名称	描述	输入/输出
DispLayer	视频输出视频层号。	输入
LayerInputPort	视频输入端口号。	输入

➤ 返回值

返回值 {

- 0 成功。
- 非 0 失败，详情参照[错误码](#)。

- 依赖
 - 头文件：mi_disp.h、mi_disp_datatype.h
 - 库文件：libmi_disp.a / libmi_disp.so

※ 注意
调用前必须使能视频层。

➤ 举例
无

➤ 相关主题
[MI_DISP_DisableInputPort](#)
[MI_DISP_PauseInputPort](#)

[MI_DISP_ResumeInputPort](#)
[MI_DISP_StepInputPort](#)
[MI_DISP_ShowInputPort](#)
[MI_DISP_HideInputPort](#)

2.26. MI_DISP_ResumeInputPort

➤ 功能

恢复指定的视频输入端口。

➤ 语法

```
MI_S32 MI_DISP_ResumeInputPort (MI\_DISP\_LAYER DispLayer, MI\_DISP\_INPUTPORT
LayerInputPort);
```

➤ 形参

参数名称	描述	输入/输出
DispLayer	视频输出视频层号。	输入
LayerInputPort	视频输入端口号。	输入

➤ 返回值

返回值 {
0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

※ 注意

调用前必须使能视频层。

➤ 举例

无

➤ 相关主题

[MI_DISP_DisableInputPort](#)
[MI_DISP_PauseInputPort](#)
[MI_DISP_ResumeInputPort](#)
[MI_DISP_StepInputPort](#)
[MI_DISP_ShowInputPort](#)
[MI_DISP_HideInputPort](#)

2.27. MI_DISP_StepInputPort

➤ 功能

单帧播放指定的视频输入端口。

➤ 语法

MI_S32 MI_DISP_StepInputPort ([MI_DISP_LAYER](#) DispLayer, [MI_DISP_INPUTPORT](#) LayerInputPort);

➤ 形参

参数名称	描述	输入/输出
DispLayer	视频输出视频层号。	输入
LayerInputPort	视频输入端口号。	输入

➤ 返回值

返回值 {
0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

※ 注意

- 目前只支持 MSR930、MSR650x。
- 调用前必须使能视频层。

➤ 举例

无

➤ 相关主题

[MI_DISP_DisableInputPort](#)
[MI_DISP_PauseInputPort](#)
[MI_DISP_ResumeInputPort](#)
[MI_DISP_StepInputPort](#)
[MI_DISP_ShowInputPort](#)
[MI_DISP_HideInputPort](#)

2.28. MI_DISP_ShowInputPort

➤ 功能

显示指定的视频输入端口。

➤ 语法

MI_S32 MI_DISP_ShowInputPort ([MI_DISP_LAYER](#) DispLayer, [MI_DISP_INPUTPORT](#) LayerInputPort);

➤ 形参

参数名称	描述	输入/输出
DispLayer	视频输出视频层号。	输入
LayerInputPort	视频输入端口号。	输入

- 返回值

返回值 {

0 成功。

非 0 失败，详情参照[错误码](#)。
- 依赖
 - 头文件：mi_disp.h、mi_disp_datatype.h
 - 库文件：libmi_disp.a / libmi_disp.so
- ※ 注意

调用前必须使能视频层。
- 举例

无
- 相关主题

[MI_DISP_DisableInputPort](#)
[MI_DISP_PauseInputPort](#)
[MI_DISP_ResumeInputPort](#)
[MI_DISP_StepInputPort](#)
[MI_DISP_ShowInputPort](#)
[MI_DISP_HideInputPort](#)

2.29. MI_DISP_HideInputPort

- 功能

隐藏指定的视频输入端口。
- 语法

MI_S32 MI_DISP_HideInputPort ([MI_DISP_LAYER](#) DispLayer, [MI_DISP_INPUTPORT](#) LayerInputPort);
- 形参

参数名称	描述	输入/输出
DispLayer	视频输出视频层号。	输入
LayerInputPort	视频输入端口号。	输入
- 返回值

返回值 {

0 成功。

非 0 失败，详情参照[错误码](#)。
- 依赖
 - 头文件：mi_disp.h、mi_disp_datatype.h
 - 库文件：libmi_disp.a / libmi_disp.so

※ 注意
调用前必须使能视频层。

➤ 举例
无

➤ 相关主题
[MI_DISP_DisableInputPort](#)
[MI_DISP_PauseInputPort](#)
[MI_DISP_ResumeInputPort](#)
[MI_DISP_StepInputPort](#)
[MI_DISP_ShowInputPort](#)
[MI_DISP_HideInputPort](#)

2.30. MI_DISP_SetInputPortSyncMode

➤ 功能
设定指定的视频输入端口同步模式。

➤ 语法
MI_S32 MI_DISP_SetInputPortSyncMode ([MI_DISP_LAYER](#) DispLayer,
[MI_DISP_INPUTPORT](#) LayerInputPort, [MI_DISP_SyncMode_e](#) eMode);

➤ 形参

参数名称	描述	输入/输出
DispLayer	视频输出视频层号。	输入
LayerInputPort	视频输入端口号。	输入
eMode	输入端口同步模式	输入

➤ 返回值
返回值 {
0 成功。
非 0 失败，详情参照[错误码](#)。
}

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

※ 注意

- 目前只支持 MSR930、MSR650x。
- 调用前必须使能视频层。

➤ 举例
无

➤ 相关主题
无

2.31. MI_DISP_QueryInputPortStat

➤ 功能

查询视频输入端口状态。

➤ 语法

```
MI_S32 MI_DISP_QueryInputPortStat(MI\_DISP\_LAYER DispLayer, MI\_DISP\_INPUTPORT
LayerInputPort, MI\_DISP\_QueryChanelStatus\_t *pstStatus);
```

➤ 形参

参数名称	描述	输入/输出
DispLayer	视频输出视频层号。	输入
LayerInputPort	视频输入端口号。	输入
pstStatus	输入端口状态指针。	输出

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

※ 注意

调用前必须使能视频层。

➤ 举例

无

➤ 相关主题

[无](#)

2.32. MI_DISP_SetZoomInWindow

➤ 功能

裁剪视频输入端口。

➤ 语法

```
MI_S32 MI_DISP_SetZoomInWindow(MI\_DISP\_LAYER DispLayer, MI\_DISP\_INPUTPORT
LayerInputPort, MI\_DISP\_VidWinRect\_t* pstZoomRect);
```

➤ 形参

参数名称	描述	输入/输出
DispLayer	视频输出视频层号。	输入
LayerInputPort	视频输入端口号。	输入
pstZoomRect	视频裁剪属性指针。	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

※ 注意

调用前必须使能视频层。

➤ 举例

无

➤ 相关主题

[MI_DISP_VidWinRect_t](#)

2.33. MI_DISP_GetVgaParam

➤ 功能

获取VGA输出设备参数。

➤ 语法

```
MI_S32 MI_DISP_GetVgaParam(MI\_DISP\_DEV DispDev, MI\_DISP\_VgaParam\_t
*pstVgaParam);
```

➤ 形参

参数名称	描述	输入/输出
DispDev	视频输出设备号。	输入
pstVgaParam	VGA 图像输出效果结构体指针。	输出

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

※ 注意
调用前必须使能视频层。

➤ 举例
无

➤ 相关主题
[MI_DISP_SetVgaParam](#)

2.34. MI_DISP_SetVgaParam

➤ 功能
设定VGA输出设备参数。

➤ 语法
MI_S32 MI_DISP_SetVgaParam ([MI_DISP_DEV](#) DispDev, [MI_DISP_VgaParam_t](#) *pstVgaParam);

➤ 形参

参数名称	描述	输入/输出
DispDev	视频输出设备号。	输入
pstVgaParam	VGA 图像输出效果结构体指针。	输入

➤ 返回值

返回值 {
 0 成功。
 非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

※ 注意
调用前必须使能视频层。

➤ 举例
无

➤ 相关主题
[MI_DISP_GetVgaParam](#)

2.35. MI_DISP_GetHdmiParam

➤ 功能
获取HDMI输出设备参数。

➤ 语法

```
MI_S32 MI_DISP_GetHdmiParam(MI\_DISP\_DEV DispDev, MI\_DISP\_HdmiParam\_t
*pstHdmiParam);
```

➤ 形参

参数名称	描述	输入/输出
DispDev	视频输出设备号。	输入
pstHdmiParam	HDMI 图像输出效果结构体指针。	输出

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

※ 注意

调用前必须使能视频层。

➤ 举例

无

➤ 相关主题

[MI_DISP_SetHdmiParam](#)

2.36. MI_DISP_SetHdmiParam

➤ 功能

设定HDMI输出设备参数。

➤ 语法

```
MI_S32 MI_DISP_SetHdmiParam(MI\_DISP\_DEV DispDev, MI\_DISP\_HdmiParam\_t
*pstHdmiParam);
```

➤ 形参

参数名称	描述	输入/输出
DispDev	视频输出设备号。	输入
pstHdmiParam	HDMI 图像输出效果结构体指针。	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

- 依赖
- 头文件：mi_disp.h、mi_disp_datatype.h
 - 库文件：libmi_disp.a / libmi_disp.so

※ 注意

调用前必须使能视频层。

➤ 举例

无

➤ 相关主题

[MI_DISP_GetHdmiParam](#)

2.37. MI_DISP_GetLcdParam

➤ 功能

获取LCD输出设备参数。

➤ 语法

```
MI_S32 MI_DISP_GetLcdParam(MI\_DISP\_DEV DispDev, MI\_DISP\_LcdParam\_t *pstLcdParam);
```

➤ 形参

参数名称	描述	输入/输出
DispDev	视频输出设备号。	输入
pstLcdParam	LCD 图像输出效果结构体指针。	输出

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

- 依赖
- 头文件：mi_disp.h、mi_disp_datatype.h
 - 库文件：libmi_disp.a / libmi_disp.so

※ 注意

调用前必须使能视频层。

➤ 举例

无

➤ 相关主题

[MI_DISP_SetLcdParam](#)

2.38. MI_DISP_SetLcdParam

➤ 功能

设定LCD输出设备参数。

➤ 语法

```
MI_S32 MI_DISP_SetLcdParam(MI_DISP_DEV DispDev, MI_DISP_LcdParam_t *pstLcdParam);
```

➤ 形参

参数名称	描述	输入/输出
DispDev	视频输出设备号。	输入
pstLcdParam	LCD 图像输出效果结构体指针。	输入

➤ 返回值

返回值 {
0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

※ 注意

调用前必须使能视频层。

➤ 举例

无

➤ 相关主题

[MI_DISP_GetLcdParam](#)

2.39. MI_DISP_GetCvbsParam

➤ 功能

获取CVBS输出设备参数。

➤ 语法

```
MI_S32 MI_DISP_GetCvbsParam(MI_DISP_DEV DispDev, MI_DISP_CvbsParam_t *pstCvbsParam);
```

➤ 形参

参数名称	描述	输入/输出
DispDev	视频输出设备号。	输入
pstCvbsParam	CVBS 图像输出效果结构体指针。	输出

➤ 返回值

{
0 成功。

返回值

非 0 失败，详情参照[错误码](#)。

- 依赖
 - 头文件：mi_disp.h、mi_disp_datatype.h
 - 库文件：libmi_disp.a / libmi_disp.so

- ※ 注意

调用前必须使能视频层。

- 举例

无

- 相关主题

[MI_DISP_SetCvbsParam](#)

2.40. MI_DISP_SetCvbsParam

- 功能

设定CVBS输出设备参数。

- 语法


```
MI_S32 MI_DISP_SetCvbsParam(MI\_DISP\_DEV DispDev, MI\_DISP\_CvbsParam\_t *pstCvbsParam);
```

- 形参

参数名称	描述	输入/输出
DispDev	视频输出设备号。	输入
pstCvbsParam	CVBS 图像输出效果结构体指针。	输入

- 返回值

返回值

{

0 成功。

非 0 失败，详情参照[错误码](#)。

- 依赖
 - 头文件：mi_disp.h、mi_disp_datatype.h
 - 库文件：libmi_disp.a / libmi_disp.so

- ※ 注意
 - 调用前必须使能视频层。

- 举例

无

- 相关主题

MI_DISP_GetCvbsParam

2.41. MI_DISP_DeviceGetColorTemperture

➤ 功能

获取图像输出色温参数。

➤ 语法

```
MI_S32 MI_DISP_DeviceGetColorTemperture(MI\_DISP\_DEV DispDev,  
MI\_DISP\_ColorTemperature\_t *pstColorTempInfo);
```

➤ 形参

参数名称	描述	输入/输出
DispDev	视频输出设备号。	输入
pstColorTempInfo	图像输出色温参数结构体指针。	输出

➤ 返回值

返回值 {
0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

※ 注意

- 目前只支持 Takoyaki。
- 调用前必须使能视频层。

➤ 举例

无

➤ 相关主题

[MI_DISP_DeviceSetColorTemperture](#)

2.42. MI_DISP_DeviceSetColorTemperture

➤ 功能

设定图像输出色温参数。

➤ 语法

```
MI_S32 MI_DISP_DeviceSetColorTemperture(MI\_DISP\_DEV DispDev,  
MI\_DISP\_ColorTemperature\_t *pstColorTempInfo);
```

➤ 形参

参数名称	描述	输入/输出
DispDev	视频输出设备号。	输入

参数名称	描述	输入/输出
pstColorTempInfo	图像输出色温参数结构体指针。	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

※ 注意

- 目前只支持 Takoyaki。
- 调用前必须使能视频层。

➤ 举例

无

➤ 相关主题

[MI_DISP_DeviceGetColorTempature](#)

2.43. MI_DISP_DeviceSetGammaParam

➤ 功能

设定图像输出Gamma参数。

➤ 语法

```
MI_S32 MI_DISP_DeviceSetGammaParam(MI\_DISP\_DEV DispDev,  
MI\_DISP\_GammaParam\_t * pstGammaParam);
```

➤ 形参

参数名称	描述	输入/输出
DispDev	视频输出设备号。	输入
pstGammaParam	图像输出 Gamma 参数结构体指针。	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

※ 注意

- 目前只支持 **Takoyaki**。
- 调用前必须使能视频层。

➤ 举例

无

➤ 相关主题

无

2.44. MI_DISP_SetVideoLayerRotateMode

➤ 功能

设定视频层旋转参数。

➤ 语法

```
MI_S32 MI_DISP_SetVideoLayerRotateMode(MI\_DISP\_LAYER DispLayer,  
MI\_DISP\_RotateConfig\_t *pstRotateConfig);
```

➤ 形参

参数名称	描述	输入/输出
DispLayer	视频输出视频层号。	输入
pstRotateConfig	图像输出旋转参数结构体指针。	输入

➤ 返回值

返回值 {
0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

※ 注意

- 目前只支持 **Takoyaki**、**Tiramisu**、**Muffin**。其中只有 **Takoyaki** 支持旋转 180 度。
- 调用前需使能视频层。
- 调用前需禁用其上全部的输入端口。
- 前级需绑定 VDEC/SCL。
- 每个视频层仅支援一个输入端口。例如，开启旋转 90° 后，每个视频层仅能启用输入端口 0。

➤ 举例

无

➤ 相关主题

无

2.45. MI_DISP_ClearInputPortBuffer

➤ 功能

清除当前显示窗口视频数据。

➤ 语法

```
MI_S32 MI_DISP_ClearInputPortBuffer(MI_DISP_LAYER DispLayer, MI_DISP_INPUTPORT
LayerInputPort, MI_BOOL bClrAll);
```

➤ 形参

参数名称	描述	输入/输出
DispLayer	视频输出视频层号。	输入
LayerInputPort	视频输入端口号。	输入
bClrAll	是否清除当前显示内容	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

※ 注意

调用前必须使能视频层。

➤ 举例

无

➤ 相关主题

无

2.46. MI_DISP_InitDev

➤ 功能

初始化 disp 设备。

➤ 语法

```
MI_S32 MI_DISP_InitDev(MI_DISP_InitParam_t *pstInitParam);
```

➤ 形参

参数名称	描述	输入/输出
pstInitParam	设备初始化参数。 暂没有用到，可以配置为 NULL	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

※ 注意

- 如果本接口在调用任意的 DISP API 之前没有调用，内部会自动初始化设备

➤ 举例

无

➤ 相关主题

无

2.47. MI_DISP_DeInitDev

➤ 功能

反初始化 disp 设备。

➤ 语法

```
MI_S32 MI_DISP_DeInitDev(void);
```

➤ 形参

无

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

※ 注意

- 此函数必须在初始化设备后调用，否则返回失败。
- 如果本接口在 app 退出前没有调用，内部会自动反初始化设备。

➤ 举例

无

- 相关主题
无

2.48. MI_DISP_SetWBCSource

- 功能

设置视频回写设备的回写源，可设置从设备回写还是从设备的视频层回写。

- 语法

MI_S32 MI_DISP_SetWBCSource(MI_DISP_WBC DispWbc, const MI_DISP_WBC_Source_t *pstWbcSource);

- 形参

参数名称	描述	输入/输出
DispWbc	回写设备号。	输入
pstWbcSource	回写源结构体指针。	输入

- 返回值

返回值 {
0 成功。
非 0 失败，详情参照[错误码](#)。

- 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

- ※ 注意

- 目前只支持 Tiramisu、Muffin、Mochi，回写设备只支持一个。
- u32SourceId 只支持 0/1。

- 举例

```
MI_DISP_WBC_Source_t stWbcSource;
MI_DISP_WBC_Attr_t stWbcAttr;

stWbcSource.eSourceType = MI_DISP_WBC_SOURCE_DEV;
stWbcSource.u32SourceId = 0;
MI_DISP_SetWBCSource(0, &stWbcSource);

stWbcAttr.stTargetSize.u32Width = 720;
stWbcAttr.stTargetSize.u32Height = 576;
stWbcAttr.ePixFormat = E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_420;
```

```
MI_DISP_SetWBCAttr(0, &stWbcAttr);
MI_DISP_EnableWBC(0);

stSrcChnPort.eModId = E_MI_MODULE_ID_WBC;
stSrcChnPort.u32DevId = 0;
stSrcChnPort.u32ChnId = 0;
stSrcChnPort.u32PortId = 0;

stDstChnPort.eModId = E_MI_MODULE_ID_DISP;
stDstChnPort.u32DevId = 1;
stDstChnPort.u32ChnId = 0;
stDstChnPort.u32PortId = 0;

MI_SYS_BindChnPort(0, &stSrcChnPort, &stDstChnPort, 30, 30);
```

➤ 相关主题

[MI_DISP_GetWBCSource](#)
[MI_DISP_SetWBCAttr](#)
[MI_DISP_GetWBCAttr](#)
[MI_DISP_EnableWBC](#)
[MI_DISP_DisableWBC](#)

2.49. MI_DISP_GetWBCSource

➤ 功能

获取视频回写设备的回写源。

➤ 语法

```
MI_S32 MI_DISP_GetWBCSource(MI\_DISP\_WBC DispWbc, MI\_DISP\_WBC\_Source\_t
*pstWbcSource);
```

➤ 形参

参数名称	描述	输入/输出
DispWbc	回写设备号。	输入
pstWbcSource	回写源结构体指针。	输出

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

※ 注意

无

➤ 举例

无

➤ 相关主题

[MI_DISP_SetWBCSource](#)
[MI_DISP_SetWBCAttr](#)
[MI_DISP_GetWBCAttr](#)
[MI_DISP_EnableWBC](#)
[MI_DISP_DisableWBC](#)

2.50. MI_DISP_SetWBCAttr

➤ 功能

设置视频回写设备属性。

➤ 语法

```
MI_S32 MI_DISP_SetWBCAttr(MI\_DISP\_WBC DispWbc, const MI\_DISP\_WBC\_Attr\_t
*pstWbcAttr);
```

➤ 形参

参数名称	描述	输入/输出
DispWbc	回写设备号。	输入
pstWbcAttr	回写设备属性结构体指针。	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

※ 注意

无

➤ 举例

无

➤ 相关主题

[MI_DISP_SetWBCSource](#)
[MI_DISP_GetWBCSource](#)
[MI_DISP_GetWBCAttr](#)

[MI_DISP_EnableWBC](#)
[MI_DISP_DisableWBC](#)

2.51. MI_DISP_GetWBCAttr

➤ 功能

获取视频回写设备属性。

➤ 语法

```
MI_S32 MI_DISP_GetWBCAttr(MI\_DISP\_WBC DispWbc, MI\_DISP\_WBC\_Attr\_t
*pstWbcAttr);
```

➤ 形参

参数名称	描述	输入/输出
DispWbc	回写设备号。	输入
pstWbcAttr	回写设备属性结构体指针。	输出

➤ 返回值

返回值 {
0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

※ 注意

无

➤ 举例

无

➤ 相关主题

[MI_DISP_SetWBCSource](#)
[MI_DISP_GetWBCSource](#)
[MI_DISP_SetWBCAttr](#)
[MI_DISP_EnableWBC](#)
[MI_DISP_DisableWBC](#)

2.52. MI_DISP_EnableWBC

➤ 功能

使能视频回写设备。

➤ 语法

```
MI_S32 MI_DISP_EnableWBC(MI\_DISP\_WBC DispWbc);
```

➤ 形参

参数名称	描述	输入/输出
DispWbc	回写设备号。	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h
- 库文件：libmi_disp.a / libmi_disp.so

※ 注意

无

➤ 举例

无

➤ 相关主题

[MI_DISP_SetWBCSource](#)
[MI_DISP_GetWBCSource](#)
[MI_DISP_SetWBCAttr](#)
[MI_DISP_GetWBCAttr](#)
[MI_DISP_DisableWBC](#)

2.53. MI_DISP_DisableWBC

➤ 功能

禁用视频回写设备。

➤ 语法

MI_S32 MI_DISP_DisableWBC([MI_DISP_WBC](#) DispWbc);

➤ 形参

参数名称	描述	输入/输出
DispWbc	回写设备号。	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_disp.h、mi_disp_datatype.h

- 库文件: libmi_disp.a / libmi_disp.so

※ 注意

无

➤ 举例

无

➤ 相关主题

[MI_DISP_SetWBCSource](#)
[MI_DISP_GetWBCSource](#)
[MI_DISP_SetWBCAttr](#)
[MI_DISP_GetWBCAttr](#)
[MI_DISP_EnableWBC](#)

3. DISP 数据类型

DISP 模块相关数据类型定义如下：

表 3-2 DISP 数据类型

数据类型	描述
MI_DISP_DEV	定义 DISP_DEV 类型
MI_DISP_LAYER	定义 DISP_LAYER 类型
MI_DISP_INPUTPORT	定义 LAYER_INPUTPORT 类型。
MI_DISP_Interface_e	定义 DISP 视频输出接口
MI_DISP_IntfSync_e	定义 DISP 视频输出 timing
MI_DISP_SyncInfo_t:	定义 DISP 视频输出时序相关参数。
MI_DISP_PubAttr_t	定义 DISP 公共属性。
MI_DISP_DevAttachMode_e	定义 DISP 视频同源输出模式。
MI_DISP_DevAttachAttr_t	定义 DISP 视频同源输出属性。
MI_DISP_Csc_t	定义 DISP 颜色转换信息
MI_DISP_CscMatrix_e	定义 DISP 颜色转换矩阵
MI_DISP_VgaParam_t:	定义 DISP VGA 输出参数信息
MI_DISP_HdmiParam_t	定义 DISP HDMI 输出参数信息。
MI_DISP_LcdParam_t	定义 DISP LCD 输出参数信息
MI_DISP_CvbsParam_t	定义 DISP CVBS 输出图像参数。
MI_DISP_ColorTemperature_t	定义 DISP 视频输出色温参数
MI_DISP_GammaParam_t	定义 DISP 视频输出 Gamma 参数
MI_DISP_VideoLayerSize_t	定义 DISP 视频层画面大小。
MI_DISP_VideoLayerAttr_t	定义 DISP 视频层属性
MI_DISP_RotateMode_e	定义 DISP 窗口属性
MI_DISP_RotateConfig_t	定义 DISP 视频旋转参数
MI_DISP_VidWinRect_t	定义 DISP 视频输出窗口信息
MI_DISP_InputPortAttr_t	定义 DISP 输入端口属性
MI_DISP_Position_t	定义 DISP 输入端口位置
MI_DISP_SyncMode_e	定义 DISP 同步模式
MI_DISP_InputPortStatus_e	定义 DISP 输入端口状态类型
MI_DISP_QueryChannelStatus_t	定义 DISP 输入端口状态信息。

数据类型	描述
MI_DISP_VideoFrame_t	定义 DISP 视频帧信息
MI_DISP_InitParam_t	定义 DISP 设备初始化参数
MI_DISP_WBC	定义 DISP_WBC 类型
MI_DISP_WBC_SourceType_e	视频回写设备捕获数据的源类型
MI_DISP_WBC_Source_t	视频回写设备回写源
MI_DISP_WBC_TargetSize_t	视频回写的目标大小
MI_DISP_WBC_Attr_t	视频回写设备属性

3.1. MI_DISP_DEV

- 说明
定义 DISP_DEV 类型。
- 定义
typedef MI_S32 MI_DISP_DEV;
- 成员
无。
- ※ 注意事项
无。

3.2. MI_DISP_LAYER

- 说明
定义 DISP LAYER 类型。
DISP 以 video layer 作为输出单位，对于有多个 Input Port 的 DISP 来说，所有 Input Port 最终拼接为一个 video layer 输出到显示器；对于只有一个 Input Port 的 DISP，Input Port size 等于 video layer size。
- 定义
typedef MI_S32 MI_DISP_LAYER;
- 成员
无。
- 相关数据类型及接口
无。

3.3. MI_DISP_INPUTPORT

- 说明
定义 LAYER_INPUTPORT 类型。

➤ 定义

```
typedef MI_S32 MI_DISP_INPUTPORT;
```

※ 注意事项

无。

➤ 相关数据类型及接口

无。

3.4. MI_DISP_Interface_e

➤ 说明

定义视频输出接口。

➤ 定义

```
typedef enum
{
    E_MI_DISP_INTF_CVBS = 0,
    E_MI_DISP_INTF_YPBPR,
    E_MI_DISP_INTF_VGA,
    E_MI_DISP_INTF_BT656,
    E_MI_DISP_INTF_BT1120,
    E_MI_DISP_INTF_HDMI,
    E_MI_DISP_INTF_LCD,
    E_MI_DISP_INTF_BT656_H,
    E_MI_DISP_INTF_BT656_L,
    E_MI_DISP_INTF_TTL,
    E_MI_DISP_INTF_MIPIDSI,
    E_MI_DISP_INTF_TTL_SPI_IF,
    E_MI_DISP_INTF_SRGB,
    E_MI_DISP_INTF_MCU,
    E_MI_DISP_INTF_MCU_NOFLM,
    E_MI_DISP_INTF_BT601,
    E_MI_DISP_INTF_BT1120_DDR,
    E_MI_DISP_INTF_MAX,
}MI_DISP_Interface_e;
```

※ 注意事项

E_MI_DISP_INTF_BT1120 为单边沿采样；E_MI_DISP_INTF_BT1120_DDR 为双边沿采样

➤ 相关数据类型及接口

[MI_DISP_PubAttr_t](#)
[MI_DISP_SetPubAttr](#)。

3.5. MI_DISP_IntfSync_e

➤ 说明

定义 DISP timing 类型。

➤ 定义

```
typedef enum
{
    E_MI_DISP_OUTPUT_PAL = 0,
    E_MI_DISP_OUTPUT_NTSC,
    E_MI_DISP_OUTPUT_960H_PAL, /* ITU-R BT.1302 960 x 576 at 50 Hz (interlaced) */
    E_MI_DISP_OUTPUT_960H_NTSC, /* ITU-R BT.1302 960 x 480 at 60 Hz (interlaced) */

    E_MI_DISP_OUTPUT_480i60,
    E_MI_DISP_OUTPUT_576i50,
    E_MI_DISP_OUTPUT_480P60,
    E_MI_DISP_OUTPUT_576P50,
    E_MI_DISP_OUTPUT_720P50,
    E_MI_DISP_OUTPUT_720P60,
    E_MI_DISP_OUTPUT_1080P24,
    E_MI_DISP_OUTPUT_1080P25,
    E_MI_DISP_OUTPUT_1080P30,
    E_MI_DISP_OUTPUT_1080I50,
    E_MI_DISP_OUTPUT_1080I60,
    E_MI_DISP_OUTPUT_1080P50,
    E_MI_DISP_OUTPUT_1080P60,

    E_MI_DISP_OUTPUT_640x480_60, /* VESA 640 x 480 at 60 Hz (non-interlaced) CVT */
    E_MI_DISP_OUTPUT_800x600_60, /* VESA 800 x 600 at 60 Hz (non-interlaced) */
    E_MI_DISP_OUTPUT_1024x768_60, /* VESA 1024 x 768 at 60 Hz (non-interlaced) */
    E_MI_DISP_OUTPUT_1280x1024_60, /* VESA 1280 x 1024 at 60 Hz (non-interlaced) */
    E_MI_DISP_OUTPUT_1366x768_60, /* VESA 1366 x 768 at 60 Hz (non-interlaced) */
    E_MI_DISP_OUTPUT_1440x900_60, /* VESA 1440 x 900 at 60 Hz (non-interlaced) CVT
Compliant */
    E_MI_DISP_OUTPUT_1280x800_60, /* 1280*800@60Hz VGA@60Hz */
    E_MI_DISP_OUTPUT_1680x1050_60, /* VESA 1680 x 1050 at 60 Hz (non-interlaced) */
    E_MI_DISP_OUTPUT_1920x2160_30, /* 1920x2160_30 */
    E_MI_DISP_OUTPUT_1600x1200_60, /* VESA 1600 x 1200 at 60 Hz (non-interlaced) */
    E_MI_DISP_OUTPUT_1920x1200_60, /* VESA 1920 x 1600 at 60 Hz (non-interlaced) CVT
(Reduced Blanking) */

    E_MI_DISP_OUTPUT_2560x1440_30, /* 2560x1440_30 */
    E_MI_DISP_OUTPUT_2560x1440_50, /* 2560x1440_50 */
    E_MI_DISP_OUTPUT_2560x1440_60, /* 2560x1440_60 */
    E_MI_DISP_OUTPUT_2560x1600_60, /* 2560x1600_60 */
    E_MI_DISP_OUTPUT_3840x2160_25, /* 3840x2160_25 */
    E_MI_DISP_OUTPUT_3840x2160_30, /* 3840x2160_30 */
    E_MI_DISP_OUTPUT_3840x2160_60, /* 3840x2160_60 */
    E_MI_DISP_OUTPUT_1920x1080_5994, /* 1920x1080_59.94 */
    E_MI_DISP_OUTPUT_1920x1080_2997, /* 1920x1080_29.97 */
    E_MI_DISP_OUTPUT_1280x720_5994, /* 1280x720_59.94 */
    E_MI_DISP_OUTPUT_1280x720_2997, /* 1280x720_29.97 */
    E_MI_DISP_OUTPUT_3840x2160_2997, /* 3840x2160_29.97 */
}
```

```
E_MI_DISP_OUTPUT_720P24,    /* 1280x720_24 */
E_MI_DISP_OUTPUT_720P25,    /* 1280x720_25 */
E_MI_DISP_OUTPUT_720P30,    /* 1280x720_30 */
E_MI_DISP_OUTPUT_1920x1080_2398, /* 1920x1080_23.98 */
E_MI_DISP_OUTPUT_3840x2160_24, /* 3840x2160_24 */
E_MI_DISP_OUTPUT_848x480_60, /* 848x480_60 */
E_MI_DISP_OUTPUT_1280x768_60, /* 1280x768_60 */
E_MI_DISP_OUTPUT_1280x960_60, /* 1280x960_60 */
E_MI_DISP_OUTPUT_1360x768_60, /* 1360x768_60 */
E_MI_DISP_OUTPUT_1400x1050_60, /* 1400x1050_60 */
E_MI_DISP_OUTPUT_1600x900_60, /* 1600x900_60 */
E_MI_DISP_OUTPUT_1920x1440_60, /* 1920x1440_60 */
```

```
E_MI_DISP_OUTPUT_USER,
E_MI_DISP_OUTPUT_MAX,
```

```
} MI_DISP_OutputTiming_e;
```

➤ 成员

各个 Timing 的定义，LCD 显示应该使用 E_MI_DISP_OUTPUT_USER。

※ 注意事项

➤ 相关数据类型及接口

[MI_DISP_PubAttr_t](#)
[MI_DISP_SetPubAttr](#)。

3.6. [MI_DISP_SyncInfo_t](#)

➤ 说明

定义 DISP 输出屏参信息。

➤ 定义

```
typedef struct MI_DISP_SyncInfo_s
{
    MI_BOOL bSynm; /* sync mode(0:timing,as BT.656; 1:signal,as LCD) */
    MI_BOOL bIop; /* interlaced or progressive display(0:i; 1:p) */
    MI_U8 u8Intfb; /* interlace bit width while output */

    MI_U16 u16VStart; /* vertical de start */
    MI_U16 u16Vact; /* vertical active area */
    MI_U16 u16Vbb; /* vertical back blank porch */
    MI_U16 u16Vfb; /* vertical front blank porch */

    MI_U16 u16HStart; /* horizontal de start */
    MI_U16 u16Hact; /* horizontal active area */
    MI_U16 u16Hbb; /* horizontal back blank porch */
    MI_U16 u16Hfb; /* horizontal front blank porch */
    MI_U16 u16Hmid; /* bottom horizontal active area */

    MI_U16 u16Bvact; /* bottom vertical active area */
    MI_U16 u16Bvbb; /* bottom vertical back blank porch */
    MI_U16 u16Bvfb; /* bottom vertical front blank porch */
}
```

```

MI_U16 u16Hpw; /* horizontal pulse width */
MI_U16 u16Vpw; /* vertical pulse width */

MI_BOOL bIdv; /* inverse data valid of output */
MI_BOOL bIhs; /* inverse horizontal synch signal */
MI_BOOL bIvs; /* inverse vertical synch signal */
MI_U32 u32FrameRate;
} MI_DISP_SyncInfo_t

```

➤ 成员

成员名称	描述
u16VStart	场有效开始。一般等于场同步信号脉宽+场同步信号后肩
u16Vact	场有效行数
u16Vbb	场同步信号后肩
u16Vfb	场同步信号前肩
u16HStart	行有效开始。一般等于行同步信号脉宽+行同步信号后肩
u16Hact	行有效像素点数
u16Hbb	行同步信号后肩
u16Hfb	行同步信号前肩
u16Hpw	行同步信号脉宽
u16Vpw	场同步信号脉宽
u32Framerate	帧率

表中未列参数暂未使用到

※ 注意事项

LCD 显示时才需要配置屏参信息。

➤ 相关数据类型及接口

[MI_DISP_PubAttr_t](#)
[MI_DISP_SetPubAttr](#)。

3.7. [MI_DISP_PubAttr_t](#)

➤ 说明

定义 DISP 公共属性。

➤ 定义

```

typedef struct MI_DISP_PubAttr_s
{
    MI_U32 u32BgColor; /* Background color of a device*/
    MI\_DISP\_Interface\_e eIntfType; /* Type of a VO interface */
    MI\_DISP\_IntfSync\_e eIntfSync; /* Type of a VO interface timing */
    MI\_DISP\_SyncInfo\_t stSyncInfo; /* Information about VO interface timings */
} MI_DISP_PubAttr_t;

```

➤ 成员

成员名称	描述
u32BgColor	设定背景颜色。
u32IntfType	接口定义： MI_DISP_Interface_e
eIntfSync	接口时序定义： MI_DISP_IntfSync_e
stSyncInfo	接口自定义时序定义： MI_DISP_SyncInfo_t

※ 注意事项

- stSyncInfo 是针对用户选择 E_MI_DISP_OUTPUT_USER，即 Timing 自定义的 Case.
- Maruko 的 u32BgColor 格式为 xBGR，其中 x, B, G, R 各占 8bit，x 是无效数据；其余芯片的 u32BgColor 格式为 YUYV，其中 Y, U, V 各占 8bit.

➤ 相关数据类型及接口

[MI_DISP_PubAttr_t](#)
[MI_DISP_SetPubAttr](#)

3.8. MI_DISP_DevAttachMode_e

➤ 说明

定义 DISP 视频同源输出模式。

➤ 定义

```
typedef enum
{
    E_MI_DISP_ATTACH_DEFAULT = 0,
    E_MI_DISP_ATTACH_OSD_MODE,
    E_MI_DISP_ATTACH_VIDEO_MODE,
}MI_DISP_DevAttachMode_e;
```

➤ 成员

成员名称	描述
E_MI_DISP_ATTACH_DEFAULT	同源输出默认模式
E_MI_DISP_ATTACH_OSD_MODE	输出叠加 OSD 的视频画面
E_MI_DISP_ATTACH_VIDEO_MODE	仅输出视频画面

※ 注意事项

无

➤ 相关数据类型及接口

[MI_DISP_DeviceAttach](#)
[MI_DISP_DevAttachAttr_t](#)

3.9. MI_DISP_DevAttachAttr_t

➤ 说明

定义 DISP 视频同源输出属性。

➤ 定义

```
typedef struct MI_DISP_DevAttachAttr_s
{
    MI_DISP_DevAttachMode_e eDevAttachMode;
}MI_DISP_DevAttachAttr_t;
```

➤ 成员

成员名称	描述
eDevAttachMode	设定视频同源输出模式。

※ 注意事项

无

➤ 相关数据类型及接口

[MI_DISP_DeviceAttach](#)

3.10. MI_DISP_Csc_t

➤ 说明

定义 DISP 颜色转换信息。

➤ 定义

```
typedef struct MI_DISP_Csc_s
{
    MI\_DISP\_CscMatrix\_e eCscMatrix;
    MI_U32 u32Luma;           /* luminance: 0 ~ 100 default: 50 */
    MI_U32 u32Contrast;       /* contrast : 0 ~ 100 default: 50 */
    MI_U32 u32Hue;           /* hue      : 0 ~ 100 default: 50 */
    MI_U32 u32Saturation;     /* saturation: 0 ~ 100 default: 50 */
} MI_DISP_Csc_t;
```

➤ 成员

成员名称	描述
eCscMatrix	颜色转换矩阵
u32Luma	亮度调节
u32Contrast	对比度调节
u32Hue	色度调节
u32Saturation	饱和度调节

※ 注意事项

- 目前只支持 MSR930、MSR650x、Taiyaki、Takoyaki。

➤ 相关数据类型及接口

[MI_DISP_GetVgaParam](#)
[MI_DISP_SetVgaParam](#)
[MI_DISP_GetHdmiParam](#)
[MI_DISP_SetHdmiParam](#)
[MI_DISP_GetLcdParam](#)
[MI_DISP_SetLcdParam](#)

3.11. [MI_DISP_CscMatrix_e](#)

➤ 说明

定义 DISP 颜色转换矩阵。

➤ 定义

```
typedef enum
{
    E_MI_DISP_CSC_MATRIX_BYPASS = 0,
    /* do not change color space */
    E_MI_DISP_CSC_MATRIX_BT601_TO_RGB_PC,
    /* change color space from BT.601 to RGB */
    E_MI_DISP_CSC_MATRIX_BT709_TO_RGB_PC,
    /* change color space from BT.709 to RGB */
    E_MI_DISP_CSC_MATRIX_RGB_TO_BT601_PC,
    /* change color space from RGB to BT.601 */
    E_MI_DISP_CSC_MATRIX_RGB_TO_BT709_PC,
    /* change color space from RGB to BT.709 */
    E_MI_DISP_CSC_MATRIX_NUM
} MI_DISP_CscMatrix_e;
```

➤ 成员

成员名称	描述
E_MI_DISP_CSC_MATRIX_BYPASS	不做颜色转换
E_MI_DISP_CSC_MATRIX_BT601_TO_RGB_PC	BT.601 到 RGB 色彩空间的 CSC 矩阵。
E_MI_DISP_CSC_MATRIX_BT709_TO_RGB_PC	BT.709 到 RGB 色彩空间的 CSC 矩阵。
E_MI_DISP_CSC_MATRIX_RGB_TO_BT601_PC	RGB 到 BT.601 色彩空间的 CSC 矩阵。
E_MI_DISP_CSC_MATRIX_RGB_TO_BT709_PC	RGB 到 BT.709 色彩空间的 CSC 矩阵。

※ 注意事项

- 目前只支持 MSR930、MSR650x、Taiyaki、Takoyaki。

➤ 相关数据类型及接口

[MI_DISP_GetVgaParam](#)
[MI_DISP_SetVgaParam](#)
[MI_DISP_GetHdmiParam](#)
[MI_DISP_SetHdmiParam](#)
[MI_DISP_GetLcdParam](#)
[MI_DISP_SetLcdParam](#)

3.12. [MI_DISP_VgaParam_t](#)

➤ 说明

定义 DISP VGA 设定参数信息。

➤ 定义

```
typedef struct MI_DISP_VgaParam_s
{
    MI\_DISP\_Csc\_t stCsc;          /* color space */
    MI_U32 u32Gain;              /* current gain of VGA signals. [0, 64). default:0x30 */
    MI_U32 u32Sharpness;
} MI_DISP_VgaParam_t;
```

➤ 成员

成员名称	描述
stCsc	色彩转换信息定义
u32Gain	信号强度增益
u32Sharpness	锐利度定义

※ 注意事项

- 目前只支持 MSR930、MSR650x、Taiyaki。

➤ 相关数据类型及接口

[MI_DISP_Csc_t](#)
[MI_DISP_GetVgaParam](#)
[MI_DISP_SetVgaParam](#)

3.13. [MI_DISP_HdmiParam_t](#)

➤ 说明

定义 DISP HDMI 设定参数信息。

➤ 定义

```
typedef struct MI_DISP_HdmiParam_s
{
    MI\_DISP\_Csc\_t stCsc;          /* color space */
    MI_U32 u32Gain;              /* current gain of HDMI signals. [0, 64). default:0x30 */
    MI_U32 u32Sharpness;
} MI_DISP_HdmiParam_t;
```

➤ 成员

成员名称	描述
stCsc	色彩转换信息定义
u32Gain	信号强度增益
u32Sharpness	锐利度定义

※ 注意事项

- 目前只支持 MSR930、MSR650x、Taiyaki。

➤ 相关数据类型及接口

[MI_DISP_Csc_t](#)
[MI_DISP_GetHdmiParam](#)
[MI_DISP_SetHdmiParam](#)

3.14. MI_DISP_LcdParam_t

➤ 说明

定义 DISP LCD 输出图像参数。

➤ 定义

```
typedef struct MI_DISP_LcdParam_s
{
    MI_DISP_Csc_t stCsc;           /* color space */
    MI_U32 u32Sharpness;
} MI_DISP_LcdParam_t;
```

➤ 成员

成员名称	描述
stCsc	色彩转换信息定义
u32Sharpness	锐利度定义

※ 注意事项

- 目前只支持 Takoyaki。

➤ 相关数据类型及接口

[MI_DISP_Csc_t](#) [MI_DISP_Csc_t](#)
[MI_DISP_GetLcdParam](#)
[MI_DISP_SetLcdParam](#)

3.15. MI_DISP_CvbsParam_t

➤ 说明

定义 DISP CVBS 输出图像参数。

➤ 定义

```
typedef struct MI_DISP_CvbsParam_s
{
    MI_DISP_Csc_t stCsc;           /* color space */
    MI_U32 u32Sharpness;
} MI_DISP_CvbsParam_t;
```

➤ 成员

成员名称	描述
stCsc	色彩转换信息定义
u32Sharpness	锐利度定义

※ 注意事项

- 无

➤ 相关数据类型及接口

[MI_DISP_Csc_t](#)
[MI_DISP_GetCvbsParam](#)
[MI_DISP_SetCvbsParam](#)

3.16. MI_DISP_ColorTemperature_t

➤ 说明

定义视频输出图像色温参数

➤ 定义

```
typedef struct
{
    MI_U16 u16RedOffset;
    MI_U16 u16GreenOffset;
    MI_U16 u16BlueOffset;

    MI_U16 u16RedColor; // 00~FF, 0x80 is no change
    MI_U16 u16GreenColor; // 00~FF, 0x80 is no change
    MI_U16 u16BlueColor; // 00~FF, 0x80 is no change
}MI_DISP_ColorTemperature_t;
```

➤ 成员

成员名称	描述
u16RedColor	色温矫正参数 R 分量
u16GreenColor	色温矫正参数 G 分量
u16BlueColor	色温矫正参数 B 分量

※ 注意事项

- 目前只支持 Takoyaki。

➤ 相关数据类型及接口

[MI_DISP_DeviceGetColorTempature](#)
[MI_DISP_DeviceSetColorTempature](#)

3.17. MI_DISP_GammaParam_t

➤ 说明

定义视频输出图像 Gamma 参数

➤ 定义

```
typedef struct
```

```
{
    MI_BOOL bEn;
    MI_U16 u16EntryNum;
    MI_U8 *pu8ColorR;
    MI_U8 *pu8ColorG;
    MI_U8 *pu8ColorB;
}MI_DISP_GammaParam_t;
```

➤ 成员

成员名称	描述
bEn	使能 Gamma 调整
u16EntryNum	Gamma table 成员个数
pu8ColorR	R table 指针
pu8ColorG	G table 指针
pu8ColorB	B table 指针

※ 注意事项

- 目前只支持 Takoyaki。

➤ 相关数据类型及接口

[MI_DISP_DeviceSetGammaParam](#)

3.18. [MI_DISP_VideoLayerSize_t](#)

➤ 说明

定义 DISP layer 层画面大小。

➤ 定义

```
typedef struct MI_DISP_VideoLayerSize_s
{
    MI_U32 u32Width;
    MI_U32 u32Height;
} MI_DISP_VideoLayerSize_t;
```

➤ 成员

成员名称	描述
u32Width	水平方向大小
u32Height	竖直方向大小

※ 注意事项

无

➤ 相关数据类型及接口

[MI_DISP_SetVideoLayerAttr](#)
[MI_DISP_GetVideoLayerAttr](#)

3.19. MI_DISP_VideoLayerAttr_t

➤ 说明

定义 DISP 视频层属性。

➤ 定义

```
typedef struct MI_DISP_VideoLayerAttr_s
{
    MI_DISP_VidWinRect_t stDispWin;    /* Display resolution */
    MI_DISP_VideoLayerSize_t stLayerSize; /* Canvas size of the video layer */
    MI_DISP_PixelFormat_e ePixelFormat; /* Pixel format of the video layer */
} MI_DISP_VideoLayerAttr_t;
```

➤ 成员

成员名称	描述
stDispWin	输入端口显示位置
stLayerSize	视频层大小
ePixelFormat	输入像素数据格式

※ 注意事项

无

➤ 相关数据类型及接口

[MI_DISP_SetVideoLayerAttr](#)
[MI_DISP_GetVideoLayerAttr](#)

3.20. MI_DISP_RotateMode_e

➤ 说明

定义 DISP 视频输出旋转角度。

➤ 定义

```
typedef enum
{
    E_MI_DISP_ROTATE_NONE,
    E_MI_DISP_ROTATE_90,
    E_MI_DISP_ROTATE_180,
    E_MI_DISP_ROTATE_270,
    E_MI_DISP_ROTATE_NUM,
} MI_DISP_RotateMode_e;
```

➤ 成员

成员名称	描述
E_MI_DISP_ROTATE_NONE	不做旋转
E_MI_DISP_ROTATE_90	顺时针旋转 90 度
E_MI_DISP_ROTATE_180	顺时针旋转 180 度
E_MI_DISP_ROTATE_270	顺时针旋转 270 度

※ 注意事项

- 目前只支持 Takoyaki、Tiramisu、Muffin。
- 只有 Takoyaki 支持旋转 180 度。

➤ 相关数据类型及接口

[MI_DISP_SetVideoLayerRotateMode](#)

3.21. [MI_DISP_RotateConfig_t](#)

➤ 说明

定义 DISP 视频输出旋转参数。

➤ 定义

```
typedef struct MI_DISP_RotateConfig_s
{
    MI_DISP_RotateMode_e eRotateMode;
}MI_DISP_RotateConfig_t;
```

➤ 成员

成员名称	描述
MI_DISP_RotateConfig_t;	配置输出视频旋转

※ 注意事项

- 目前只支持 Takoyaki、Tiramisu、Muffin。
- 只有 Takoyaki 支持旋转 180 度。

➤ 相关数据类型及接口

[MI_DISP_SetVideoLayerRotateMode](#)

3.22. [MI_DISP_VidWinRect_t](#)

➤ 说明

定义 DISP 窗口属性。

➤ 定义

```
typedef struct MI_DISP_VidWinRect_s
{
    MI_U16 u16X;
    MI_U16 u16Y;
    MI_U16 u16Width;
    MI_U16 u16Height;
} MI_DISP_VidWinRect_t;
```

➤ 成员

成员名称	描述
u16X	水平方向起点
u16Y	竖直方向起点
u16Width	水平方向大小
u16Height	竖直方向大小

※ 注意事项

- 每个窗口位置的 x, y, width, height 须 2pixels 对齐
- 裁剪不能超出源区域

➤ 相关数据类型及接口

[MI_DISP_SetZoomInWindow](#)

3.23. [MI_DISP_InputPortAttr_t](#)

➤ 说明

定义 DISP 输入端口属性。

➤ 定义

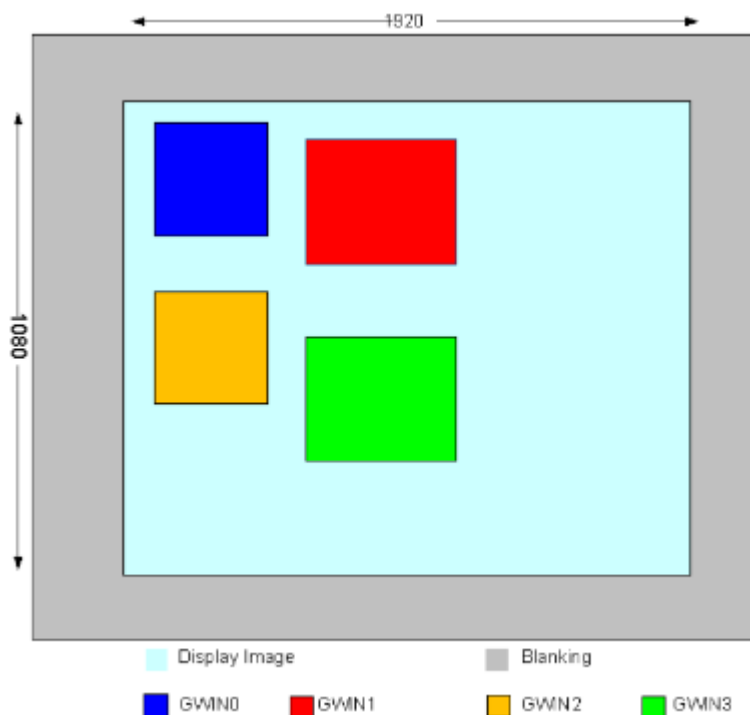
```
typedef struct MI_DISP_InputPortAttr_s
{
    MI_DISP_VidWinRect_t stDispWin;           /* rect of video out chn */
    MI_U16 u16SrcWidth;
    MI_U16 u16SrcHeight;
    MI_SYS_CompressMode_e eDecompressMode;
} MI_DISP_InputPortAttr_t;
```

➤ 成员

成员名称	描述
stDispWin	指定输入端口输出窗口位置及尺寸
u16SrcWidth	指定端口输入图像宽
u16SrcHeight	指定端口输入图像高
eDecompressMode	指定端口解压缩模式

※ 注意事项

- Takoyaki/Taiyaki/Tiramisu/Muffin/Mochi 系列芯片可以对每个 port 做 scaling up（不支持 scaling down），H/V 方向最大放大倍数为 16 倍。
- 如果芯片不支持独立对 port 做 scaling up，使用时注意 u16SrcWidth 要等于 stDispWin 中定义的 u16Width，u16SrcHeight 要等于 stDispWin 中定义的 u16Height。
- 对于有多个 input port 的 video layer 来说，每个 port 在 video layer 上的显示位置可以任意设置。
- Input port 的显示位置不能与已 Enable（包括 Hide）的 port 相互叠加，但已 Disable 的 port 可以叠加。
- 每个 window 的 x, y, width, height 须 2pixels 对齐。
- 关于 eDecompressMode，Mochi 系列芯片支持 E_MI_SYS_COMPRESS_MODE_NONE/E_MI_SYS_COMPRESS_MODE_TO_6BIT，其他 chip 仅支持 E_MI_SYS_COMPRESS_MODE_NONE。



➤ 相关数据类型及接口

[MI_DISP_SetInputPortAttr](#)

[MI_DISP_GetInputPortAttr](#)

3.24. [MI_DISP_Position_t](#)

➤ 说明

定义 DISP 输入端口位置。

➤ 定义

```
typedef struct MI_DISP_Position_s
{
    MI_U16 u16X;
    MI_U16 u16Y;
} MI_DISP_Position_t;
```

➤ 成员

成员名称	描述
u16X	水平方向起始点
u16Y	竖直方向起始点

※ 注意事项

无

➤ 相关数据类型及接口

[MI_DISP_GetInputPortDispPos](#)

[MI_DISP_SetInputPortDispPos](#)

3.25. [MI_DISP_SyncMode_e](#)

➤ 说明

定义 DISP 同步模式。

➤ 定义

```
typedef enum
{
    E_MI_DISP_SYNC_MODE_INVALID = 0,
    E_MI_DISP_SYNC_MODE_CHECK_PTS,
    E_MI_DISP_SYNC_MODE_FREE_RUN,
    E_MI_DISP_SYNC_MODE_NUM,
} MI_DISP_SyncMode_e;
```

➤ 成员

成员名称	描述
E_MI_DISP_SYNC_MODE_INVALID	无效的同步模式
E_MI_DISP_SYNC_MODE_CHECK_PTS	检查 PTS 的同步模式
E_MI_DISP_SYNC_MODE_FREE_RUN	不检查 PTS 的同步模式

※ 注意事项

- 目前只支持 MSR930、MSR650x

➤ 相关数据类型及接口

[MI_DISP_SetInputPortSyncMode](#)

3.26. [MI_DISP_InputPortStatus_e](#)

➤ 说明

定义 DISP 输入端口状态类型。

➤ 定义

```
typedef enum
{
    E_MI_DISP_INPUTPORT_STATUS_INVALID = 0,
    E_MI_DISP_INPUTPORT_STATUS_PAUSE,
    E_MI_DISP_INPUTPORT_STATUS_RESUME,
    E_MI_DISP_INPUTPORT_STATUS_STEP,
    E_MI_DISP_INPUTPORT_STATUS_REFRESH,
    E_MI_DISP_INPUTPORT_STATUS_SHOW,
    E_MI_DISP_INPUTPORT_STATUS_HIDE,
    E_MI_DISP_INPUTPORT_STATUS_NUM,
} MI_DISP_InputPortStatus_e;
```

➤ 成员

成员名称	描述
E_MI_DISP_INPUTPORT_STATUS_INVALID	无效的状态模式
E_MI_DISP_INPUTPORT_STATUS_PAUSE	暂停状态

E_MI_DISP_INPUTPORT_STATUS_RESUME	回复状态
E_MI_DISP_INPUTPORT_STATUS_STEP	步进状态
E_MI_DISP_INPUTPORT_STATUS_REFRESH	刷新状态
E_MI_DISP_INPUTPORT_STATUS_SHOW	显示状态
E_MI_DISP_INPUTPORT_STATUS_HIDE	隐藏状态

※ 注意事项
无

➤ 相关数据类型及接口
无

3.27. [MI_DISP_QueryChannelStatus_t](#)

➤ 说明
定义 DISP 输入端口状态信息。

➤ 定义

```
typedef struct MI_DISP_QueryChannelStatus_s
{
    MI_BOOL bEnable;
    MI\_DISP\_InputPortStatus\_e eStatus;
} MI_DISP_QueryChannelStatus_t;
```

➤ 成员

成员名称	描述
bEnable	输入端口使能状态
eStatus	输入端口状态类型

※ 注意事项
无

➤ 相关数据类型及接口
[MI_DISP_InputPortStatus_e](#)
[MI_DISP_QueryInputPortStat](#)

3.28. [MI_DISP_VideoFrame_t](#)

➤ 说明
定义 DISP 视频帧信息。

➤ 定义

```
typedef struct MI_DISP_VideoFrame_s
{
    MI_U32      u32Width;
    MI_U32      u32Height;
    MI_DISP_PixelFormat_e ePixelFormat;
```

```

    MI_PHY    aphyAddr[3];
    Void *    *pavirAddr[3];
    MI_U64    u64pts;
    MI_U32    u32PrivateData;
} MI_DISP_VideoFrame_t;

```

➤ 成员

成员名称	描述
u32Width	图像宽度
u32Height	图像高度
ePixelFormat	像素点格式
u32PhyAddr	图像数据物理地址
pVirAddr	图像数据虚拟地址
u64pts	图像数据时间戳
u32PrivateData	私有数据

※ 注意事项

- 目前只支持 MSR930、MSR650x

➤ 相关数据类型及接口

[MI_DISP_GetScreenFrame](#)
[MI_DISP_ReleaseScreenFrame](#)

3.29. MI_DISP_InitParam_t

➤ 说明

DISP 设备初始化参数。

➤ 定义

```

typedef struct MI_DISP_InitParam_s
{
    MI_U32 u32DevId;
    MI_U8 *u8Data;
} MI_DISP_InitParam_t;

```

➤ 成员

成员名称	描述
u32DevId	设备 ID
u8Data	数据指针 buffer

※ 注意事项

无

➤ 相关数据类型及接口

[MI_DISP_InitDev](#)

3.30. MI_DISP_WBC

➤ 说明

定义 DISP_WBC 类型。

➤ 定义

```
typedef MI_S32 MI_DISP_WBC
```

➤ 成员

无。

※ 注意事项

无。

3.31. MI_DISP_WBC_SourceType_e

➤ 说明

视频回写设备捕获数据的源类型，用于区别是从视频层回写还是从设备回写。

➤ 定义

```
typedef enum {
    MI_DISP_WBC_SOURCE_DEV,
    MI_DISP_WBC_SOURCE_VIDEO,
}MI_DISP_WBC_SourceType_e;
```

➤ 成员

成员名称	描述
MI_DISP_WBC_SOURCE_DEV	回写源是设备（video+UI）
MI_DISP_WBC_SOURCE_VIDEO	回写源是视频层（video only）

※ 注意事项

无

➤ 相关数据类型及接口

[MI_DISP_SetWBCSource](#)

[MI_DISP_GetWBCSource](#)

3.32. MI_DISP_WBC_Source_t

➤ 说明

视频回写设备回写源。

➤ 定义

```
typedef struct MI_DISP_WBC_Source_s
{
    MI_DISP_WBC_SourceType_e eSourceType;
    MI_U32 u32SourceId;
}MI_DISP_WBC_Source_t;
```

➤ 成员

成员名称	描述
eSourceType	回写源类型
u32SourceId	回写源的设备 ID

※ 注意事项

无

➤ 相关数据类型及接口

[MI_DISP_SetWBCSource](#)

[MI_DISP_GetWBCSource](#)

3.33. MI_DISP_WBC_TargetSize_t

➤ 说明

视频回写设备的回写目标大小。

➤ 定义

```
typedef struct MI_DISP_WBC_TargetSize_s
{
    MI_U32 u32Width;
    MI_U32 u32Height;
}MI_DISP_WBC_TargetSize_t;
```

➤ 成员

成员名称	描述
u32Width	回写目标的宽
u32Height	回写目标的高

※ 注意事项

WBC支持缩小，不支持放大。如果配置目标分辨率比源分辨率小，那么会自动缩小。

➤ 相关数据类型及接口

[MI_DISP_SetWBCAttr](#)

[MI_DISP_GetWBCAttr](#)

3.34. MI_DISP_WBC_Attr_t

➤ 说明

视频回写设备属性。

➤ 定义

```
typedef struct MI_DISP_WBC_Attr_s
{
    MI_DISP_WBC_TargetSize_t stTargetSize;
    MI_SYS_PixelFormat_e ePixelFormat;
```

```
MI_SYS_CompressMode_e eCompressMode;  
}MI_DISP_WBC_Attr_t;
```

➤ 成员

成员名称	描述
stTargetSize	回写目标大小
ePixFormat	回写图像的像素格式
eCompressMode	压缩模式

※ 注意事项

eDecompressMode 只有 Mochi 系列芯片时才支持。

➤ 相关数据类型及接口

[MI_DISP_SetWBCAttr](#)

[MI_DISP_GetWBCAttr](#)

4. DISP 错误码

DISP API 错误码如下表所示。

表 4-3 DISP API 错误码

错误代码	宏定义	描述
0xA00F2001	MI_ERR_DISP_INVALID_DEVID	设备 ID 超出合法范围
0xA00F2002	MI_ERR_DISP_INVALID_CHNID	输入端口 ID 超出合法范围
0xA00F2003	MI_ERR_DISP_ILLEGAL_PARAM	参数超出合法范围
0xA00F2006	MI_ERR_DISP_NULL_PTR	函数参数中有空指针
0xA00F2008	MI_ERR_DISP_NOT_SUPPORT	不支持的操作
0xA00F2009	MI_ERR_DISP_NOT_PERMIT	操作不允许
0xA00F200C	MI_ERR_DISP_NO_MEM	内存不足
0xA00F2010	MI_ERR_DISP_SYS_NOTREADY	系统未初始化
0xA00F2012	MI_ERR_DISP_BUSY	资源忙
0xA00F2040	MI_ERR_DISP_DEV_NOT_CONFIG	设备未配置
0xA00F2041	MI_ERR_DISP_DEV_NOT_ENABLE	设备未使能
0xA00F2042	MI_ERR_DISP_DEV_ALREADY_ENABLED	设备已使能
0xA00F2043	MI_ERR_DISP_DEV_ALREADY_BOUND	设备已被绑定
0xA00F2044	MI_ERR_DISP_DEV_NOT_BIND	设备未被绑定
0xA00F2045	MI_ERR_DISP_LAYER_NOT_ENABLE	视频层未使能
0xA00F2046	MI_ERR_DISP_LAYER_NOT_DISABLE	视频层未禁止
0xA00F2047	MI_ERR_DISP_LAYER_NOT_CONFIG	视频层未配置
0xA00F2048	MI_ERR_DISP_INPUTPORT_NOT_DISABLE	输入端口未禁止
0xA00F2049	MI_ERR_DISP_INPUTPORT_NOT_ENABLE	输入端口未使能
0xA00F204A	MI_ERR_DISP_INPUTPORT_NOT_CONFIG	输入端口未配置
0xA00F204B	MI_ERR_DISP_INPUTPORT_NOT_ALLOC	输入端口未分配资源
0xA00F204C	MI_ERR_DISP_INVALID_PATTERN	无效样式
0xA00F204D	MI_ERR_DISP_INVALID_POSITION	无效位置
0xA00F204E	MI_ERR_DISP_WAIT_TIMEOUT	等待超时
0xA00F204F	MI_ERR_DISP_INVALID_VIDEO_FRAME	无效视频帧
0xA00F2050	MI_ERR_DISP_INVALID_RECT_PARA	无效矩形参数
0xA00F2051	MI_ERR_DISP_INPUTPORT_SHOW_AREA_OVERLAP	输入端口区域重叠
0xA00F2052	MI_ERR_DISP_INVALID_LAYERID	视频层号超出范围

错误代码	宏定义	描述
0xA00F2053	MI_ERR_DISP_LAYER_ALREADY_BOUND	视频层已绑定
0xA00F2054	MI_ERR_DISP_LAYER_NOT_BIND	视频层未绑定

5. PROCFS 介绍

5.1. DISP cat

➤ 调试信息

```
# cat /proc/mi_modules/mi_disp/mi_disp0
```

===== disp dev0 info =====													
DevStatus	1	IrqNum	52	IrqCnt	700	BgColor	800080						
Interface	HDMI	DevTiming	720P60	CscMatrix	3	Luma	50	Contrast	50	Hue	50	Saturation	50
										Sharpness	0	Gain	0
===== disp layer[0] Info =====													
LayerId	0	BindedDevID	0	LayerWidth	0	LayerHeight	0						
LayerId	0	LayDispWidth	1280	LayDispHeight	720	Toleration	0	rotatemode	NONE				
PortId	0	enable	1	CurStatus	0	src_w	200	src_h	120	crop_x	0	crop_y	0
	1		1		0		200		120		0		0
												show_x	0
												show_y	0
												show_w	640
												show_h	720
PortId	0	RecvBufCnt	539	RecvBuf_W	200	RecvBuf_H	120	Content_W	200	Content_H	120	RecvBufStride	208
	1		527		200		120		200		120	PixFmt	semiplanar_420
												syncmode	Invalid
PortId	0	OnScreenTask	c2c5f940	FiredTask	(null)	StepTaskCnt	0	bClearAllTask	0	fps	25		
	1		c2c5f580		c2c5fac0		0		0		25		

➤ 调试信息分析

记录当前 DISP 的使用状况以及 device 属性、layer 属性、inputport 属性，可以动态地获取到这些信息，方便调试和测试。

➤ 参数说明

参数		描述
device info	DevStatus	使能或者禁用 0: 禁用 1: 使能
	IrqNum	中断号
	IrqCnt	中断发生次数
	BgColor	背景色（YUYV 各占 8bit）
	Interface	接口类型； 取值范围：（CVBS、YPBPR、VGA、BT656、BT1120、MCU、MCU_NOFLM、HDMI、LCD、BT656_H、BT656_L、TTL、MIPI_DSI、TTL_SPI、SRGB）
	DevTiming	输出时序 取值范围： [E_MI_DISP_OUTPUT_PAL~ E_MI_DISP_OUTPUT_USER]
	CscMatrix	CSC 矩阵选择 取值范围： [E_MI_DISP_CSC_MATRIX_BYPASS~ E_MI_DISP_CSC_MATRIX_RGB_TO_BT709_PC]
device info	Luma	亮度； 取值范围： [0 ~ 99] 默认 50
	Contrast	对比度； 取值范围： [0 ~ 99] 默认 50
	Hue	色调；

参数		描述
		取值范围: [0 ~ 99] 默认 50
	Saturation	饱和度 取值范围: [0 ~ 99] 默认 50
	Sharpness	锐度; 取值范围: [0 ~ 255] 默认 0
	Gain	信号增益 取值范围: [0 ~ 255] 默认 0
layer info	LayerId	layer ID 取值范围根据芯片规格而定
	BindedDevID	绑定的 device ID 取值范围根据芯片规格而定
	LayerWidth	Layer 的宽
	LayerHeight	Layer 的高
	LayDispWidth	Layer 显示的宽
	LayDispHeight	Layer 显示的高
	Toleration	PTS 误差允许阈值, 单位毫秒
	rotatemode	旋转模式 取值范围: [E_MI_DISP_ROTATE_NONE~E_MI_DISP_ROTATE_270]
port info	PortId	port ID 取值范围根据芯片规格而定
	enable	使能或者禁用 0: 禁用 1: 使能
	CurStatus	port 状态 取值范围: [invalid、pause、resume、step、refresh、show、hide] 默认是 invalid
	src_w	原始图像宽度
	src_h	原始图像高度
	crop_x	裁剪区域起始横坐标
	crop_y	裁剪区域起始纵坐标
	crop_w	裁剪区域宽度
	crop_h	裁剪区域高度
	show_x	该 port 在 layer 上的起始横坐标
	show_y	该 port 在 layer 上的起始纵坐标
	show_w	该 port 显示的宽度
	show_h	该 port 显示的高度
	RecvBufCnt	当前接收到 buff 的总量
	RecvBuf_W	输入的图像宽
	RecvBuf_H	输入的图像高
	Content_W	输入图像有效宽
	Content_H	输入图像有效高
	RecvBufStride	输入图像的 Stride

参数	描述
PixFmt	输入图像像素格式
syncmode	Check PTS/Free Run 取值范围: [Invalid、CheckPts、FreeRun]
OnScreenTask	当前正在显示的 Task, 如果为 NULL, 有可能是没有送 buff 到 disp、或 disp 没中断、或 disp 的 cmdq 没生效
FiredTask	即将要显示的 Task
StepTaskCnt	步进处理的 Task 数量
bClearAllTask	是否丢弃所有 Task 0: 不丢弃 1: 丢弃所有 Task
fps	帧率

5.2. DISP echo

功能	获取当前 DISP 所支持的 echo 命令及其具体使用形式
命令	echo help > /proc/mi_modules/mi_disp/mi_disp0
参数说明	无
举例	echo help > /proc/mi_modules/mi_disp/mi_disp0

功能	获取当前屏幕数据(保留, 目前不支持)
命令	echo getcapframe [devid] [layerid] [path] > /proc/mi_modules/mi_disp/mi_disp0
参数说明	[devid] 设备 id [layerid] 视频层 id [path] 路径及文件名
举例	echo getcapframe 0 0 /mnt/frame.yuv > /proc/mi_modules/mi_disp/mi_disp0

功能	打开/关闭每个 port 的 pts 检查
命令	echo checkframepts [layerid] [protid] [ON/OFF] > /proc/mi_modules/mi_disp/mi_disp0
参数说明	[layerid] 视频层 id [portid] port id [ON/OFF] ON 打开 OFF 关闭
举例	echo checkframepts 0 0 ON > /proc/mi_modules/mi_disp/mi_disp0

功能	dump frame data
命令	echo dumpframe [layerid] [portid] [path] > /proc/mi_modules/mi_disp/mi_disp0
参数说明	[layerid] 视频层 id [portid] port id [path] dump 出来的数据的存储路径

举例	echo dumpframe 0 0 /mnt/ > /proc/mi_modules/mi_disp/mi_disp0
----	--

功能	stop disp 单个 port get buff
命令	echo stopgetbuff [layerid] [portid] [ON/OFF] > /proc/mi_modules/mi_disp/mi_disp0
参数说明	[layerid] 视频层 id [portid] port id [ON/OFF] ON:stop OFF:resume
举例	echo stopgetbuff 0 0 ON > /proc/mi_modules/mi_disp/mi_disp0

功能	设置背景色
命令	echo bgcolor [devid] [value] > /proc/mi_modules/mi_disp/mi_disp0
参数说明	[devid] 设备 id [value] 颜色值
举例	echo setbgcolor 0 255 > /proc/mi_modules/mi_disp/mi_disp0

功能	动态设置画面效果
命令	echo csc [devid] [CscMatrix] [Contrast] [Hue] [Luma] [Saturation] [Sharpness] [Gain] > /proc/mi_modules/mi_disp/mi_disp0
参数说明	[devid] 设备 id [CscMatrix] 颜色矩阵 [Contrast] 对比度 [Hue] 色调 [Luma] 亮度 [Saturation] 饱和度 [Sharpness] 锐利度 [Gain] 增益
举例	echo csc 0 0 50 50 50 50 0 0 > /proc/mi_modules/mi_disp/mi_disp0

功能	色温调试
命令	echo colortemp [devid] [BlueOffset] [GreenOffset] [RedOffset] [BlueColor] [GreenColor] [RedColor] > /proc/mi_modules/mi_disp/mi_disp0
参数说明	[devid]当前使用的 dev id [Blue/Green/Redoffset]没有用到 [RedColor] R 分量（0-255） [GreenColor] G 分量（0-255） [BlueColor]B 分量（0-255）
举例	echo setcolortemp 0 50 50 50 50 50 50 > /proc/mi_modules/mi_disp/mi_disp0

功能	对 layer 进行旋转设置
----	----------------

命令	echo rotate [layerid] [0/1/2]> /proc/mi_modules/mi_disp/mi_disp0
参数说明	[layerid] 视频层 id [0/1/2] rotate 配置: 0: E_MI_DISP_ROTATE_NONE,不旋转 1: E_MI_DISP_ROTATE_90,旋转 90° 2: E_MI_DISP_ROTATE_270,旋转 270°
举例	echo rotate 1 1 > /proc/mi_modules/mi_disp/mi_disp0

功能	对某个 port 的原始数据进行裁剪
命令	echo crop [layerid] [portid] [x] [y] [width] [height]> /proc/mi_modules/mi_disp/mi_disp0
参数说明	[layerid] 视频层 id [portid] port id [x] 裁剪区域起始横坐标 [y] 裁剪区域起始纵坐标 [width] 裁剪区域宽度 [height] 裁剪区域高度
举例	echo crop 0 0 0 0 100 100 > /proc/mi_modules/mi_disp/mi_disp0

功能	对某个 port 进行隐藏
命令	echo hide [layerid] [portid] > /proc/mi_modules/mi_disp/mi_disp0
参数说明	[layerid] 视频层 id [portid] port id
举例	echo hide 0 0 > /proc/mi_modules/mi_disp/mi_disp0

功能	对某个 port 进行显示
命令	echo show [layerid] [portid] > /proc/mi_modules/mi_disp/mi_disp0
参数说明	[layerid] 视频层 id [portid] port id
举例	echo show 0 0 > /proc/mi_modules/mi_disp/mi_disp0

功能	对某个 port 停止获取数据
命令	echo pause [layerid] [portid] > /proc/mi_modules/mi_disp/mi_disp0
参数说明	[layerid] 视频层 id [portid] port id
举例	echo pause 0 0 > /proc/mi_modules/mi_disp/mi_disp0

功能	对某个 port 恢复，继续获取数据
----	--------------------

命令	echo resume [layerid] [portid] > /proc/mi_modules/mi_disp/mi_disp0
参数说明	[layerid] 视频层 id [portid] port id
举例	echo resume 0 0 > /proc/mi_modules/mi_disp/mi_disp0

功能	控制某个 port 获取一帧数据后停止获取数据
命令	echo step [layerid] [portid] > /proc/mi_modules/mi_disp/mi_disp0
参数说明	[layerid] 视频层 id [portid] port id
举例	echo step 0 0 > /proc/mi_modules/mi_disp/mi_disp0

功能	对某个 port 的数据进行清空
命令	echo clear [layerid] [portid] > /proc/mi_modules/mi_disp/mi_disp0
参数说明	[layerid] 视频层 id [portid] port id
举例	echo clear 0 0 > /proc/mi_modules/mi_disp/mi_disp0

5.3. WBC cat

➤ 调试信息

cat /proc/mi_modules/mi_disp/mi_wbc0

=====wbc0=====					
irqnum	isrcnt	irq enable			
55	1198	1			
src dev	srcw	srch	src type	targetW	targetH
1	1920	1080	1	200	120
OnScreenTask	FiredTask	fps			
c3a57c38	c3a57438	20			

➤ 调试信息分析

记录当前 WBC 的使用状况以及数据属性，可以动态地获取到这些信息，方便调试和测试。

➤ 参数说明

参数	描述
device info	irqnum
	Isrcnt
	irq enable
	src dev
	srcw
	中断号
	中断发生次数
	中断状态： 0：禁用 1：使能
	src 设备号
	数据源宽度

参数		描述
	srch	数据源高度
	src type	数据源类型 0: video+ui 1: video
	targetW	数据输出宽度
	targetH	数据输出高度
	OnScreenTask	当前正在输出的 Task
	FiredTask	即将要输出的 Task

5.4. WBC echo

功能	获取当前 WBC 所支持的 echo 命令及其具体使用形式
命令	echo help > /proc/mi_modules/mi_disp/mi_wbc0
参数说明	无
举例	echo help > /proc/mi_modules/mi_disp/mi_wbc0

功能	dump frame data
命令	echo dumpframe [path] > /proc/mi_modules/mi_disp/mi_wbc0
参数说明	[path] dump 出的 frame 的存储路径
举例	echo dumpframe /mnt/ > /proc/mi_modules/mi_disp/mi_wbc0